

AN ARCHITECTURE WALKTHROUGH

Inside pi-mono

Architecture, design decisions, and little bits of intuition you might not know about the internals.

Companion to github.com/badlogic/pi-mono
Written from a deep read of the source, April 2026.

Contents

— Preface

PART I · FOUNDATIONS

01 What pi-mono is

02 The shape of the monorepo

03 Design philosophy

PART II · THE FOUR FOUNDATION PACKAGES

04 pi-ai — one API, many providers

05 pi-agent-core — an agent is just a loop

06 pi-tui — a terminal that doesn't flicker

07 pi-coding-agent — the thing you actually run

PART III · CROSS-CUTTING DESIGN

08 Event streams and backpressure

09 Tool calling end-to-end

10 Sessions as JSONL trees

11 Context compaction

12 Extensions: trust, lifecycle, API

13 Auth, credentials, and OAuth

14 Three ways to run the agent

PART IV · THE ECOSYSTEM

15 pi-mom — the agent on Slack

16 pi-web-ui — agents in the browser

17 pi-pods — rolling your own vLLM

PART V · GOING FURTHER

18 Adding a new LLM provider

19 Testing agents with the faux provider

20 Release engineering

— Appendix A — Glossary

— Appendix B — File map

— Appendix C — The rules pi-mono seems to follow

Preface

I read pi-mono the same way I read any interesting codebase: I start in the README, jump to the entry points, and then chase the interesting nouns — `Agent`, `Context`, `Session`, `Tool` — until I understand what the authors were actually thinking. This book is the notes from one of those passes, cleaned up and put in an order that should be readable on its own.

Who this is for

- People **using** pi as an end-user who want to understand what's happening under the hood.
- People **building** on top of pi-mono — custom extensions, custom providers, embedding `pi-agent-core` in another product, or porting pieces of `pi-tui` into a non-pi tool.
- People **learning** how modern agent systems are actually wired. pi-mono is small, honest, and readable — a better study subject than a 50-service cloud stack.

What I'm assuming

You've written some TypeScript. You know what a REPL is. You've used at least one LLM SDK (OpenAI, Anthropic, Gemini, whatever). You don't need to know anything about terminal escape sequences, JSONL, or agent loops going in.

How to read it

Linearly is fine. Each chapter is short and self-contained enough that jumping straight to chapter 10 ("Sessions as JSONL trees") or chapter 6 ("pi-tui") also works.

Code references point at specific files, sometimes with line numbers. Line numbers are a snapshot — code moves — but the landmarks they point to (a function name, a type definition, a particular `if` branch) should stay recognizable.

What this book is *not*

It's not the API reference. That's what [the package READMEs](#) are for.

It's not a tutorial. You will learn things by reading it, but nobody is going to hold your hand through "run `npm install` and see what happens".

It's not a marketing brochure. If I think a design decision is weird or a tradeoff is uncomfortable, I say so.

A note on names

Every package in the repo is prefixed with `pi -` and published under the `@mariozechner` npm scope. Inside the repo the packages live as `packages/ai`, `packages/agent`, `packages/tui`, etc. — so `pi-agent-core` lives at `packages/agent`, which tripped me up a few times. This book uses whichever name is clearest in context and tries to be consistent inside each chapter.

Let's go.

P A R T I

Foundations

What pi-mono is

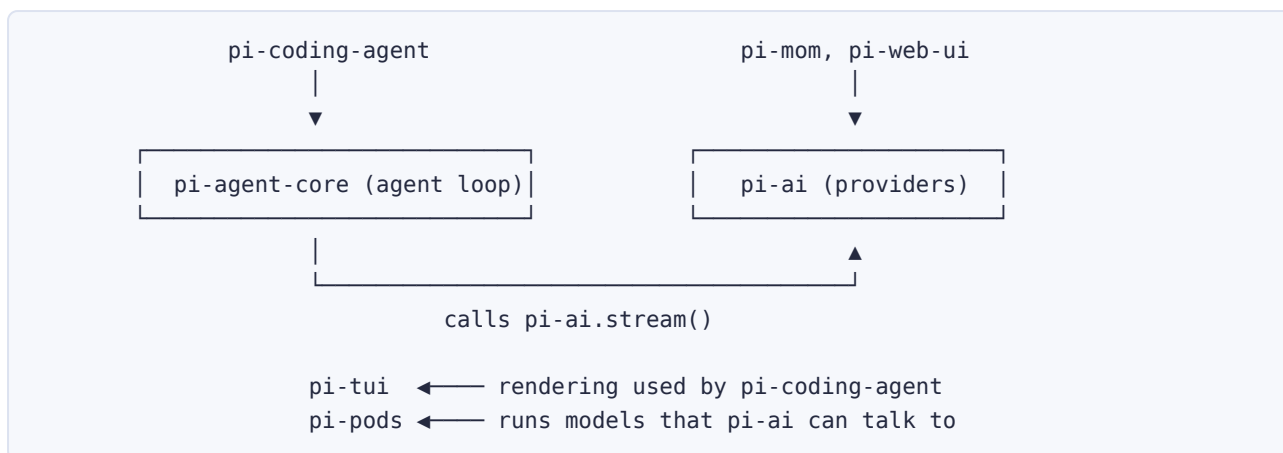
If you squint at pi-mono from a distance, it looks like a coding agent. `pi` — the CLI at `packages/coding-agent` — is the user-facing product, the thing you install with `npx @mariozechner/pi-coding-agent`, the thing you run in a terminal to get something that reads, writes, and executes code on your behalf.

That's fine as a one-line description, but it's not what the repo actually is. The repo is seven packages, and only one of them is the agent you run. The other six exist because the author had to build them to make the agent work — and along the way noticed each one was useful on its own.

The seven packages

package	directory	rough size	role
pi-ai	<code>packages/ai</code>	~16 k LOC	One streaming API in front of 20+ LLM providers.
pi-agent-core	<code>packages/agent</code>	~650 LOC	The agent loop: queue messages, call model, run tools, repeat.
pi-tui	<code>packages/tui</code>	~8 k LOC	A terminal UI toolkit with differential rendering and overlays.
pi-coding-agent	<code>packages/coding-agent</code>	~20 k LOC	The CLI user, three modes, sessions, extensions, skills, the whole product.
pi-mom	<code>packages/mom</code>	~2 k LOC	A Slack bot that delegates to the coding agent.
pi-web-ui	<code>packages/web-ui</code>	~5 k LOC	Lit-based chat components for browsers.
pi-pods	<code>packages/pods</code>	~3 k LOC	CLI to spin up vLLM on rented GPUs.

The boundaries are not arbitrary. There is a clear stack:



Each package's upstream dependency is small and deliberate:

- **pi-ai** depends on nothing pi-specific. You can use it to stream from Claude without ever touching an agent.
- **pi-agent-core** depends on pi-ai, adds a runtime. You can use it to build a custom agent without touching a terminal.
- **pi-tui** depends on nothing pi-specific. You can use it to build a fancy CLI without touching agents.
- **pi-coding-agent** depends on all three and adds the product.

Two packages (pi-ai, pi-tui) are honest standalone libraries that happen to share a scope. The other two (pi-agent-core, pi-coding-agent) are where "pi" actually lives.

Who runs it

The primary user is the author, `badlogic` — this is his daily driver for coding work. Sessions are published as training data on [Hugging Face](#), which is an unusually honest thing for an open-source agent to do. You can watch real sessions, including the failed ones.

The secondary user is anyone who wants:

- A coding agent they can actually hack on. No MCP, no cloud account, no sub-agent frameworks to learn. The whole thing is TypeScript in one repo.
- A minimal LLM client that isn't tied to a framework like LangChain. If you just want `stream({ model, context })` and no opinions about agents, prompt templates, or chains — `pi-ai` is that.
- An embeddable agent runtime. If you're building a product that wraps LLM+tools, `pi-agent-core` is small enough to read in an afternoon.

What it is not

Worth being explicit about a few things pi-mono is *not*, because the absences shape the design:

- **Not an MCP client.** The README calls this out — MCP is implemented "as extensions, not built-in". The rationale is in chapter 3.

- **Not a plan / reflect / subagent framework.** There is no `plan()` phase, no "reflection" step, no subagents by default. If you want those, you add an extension.
- **Not a browser agent, not a "computer use" agent.** The built-in tools are `read`, `write`, `edit`, `bash`, and a few grep/find helpers. Anything else is an extension.
- **Not a framework.** There's no plugin marketplace, no config DSL, no YAML other than the frontmatter in a skill file. "Configuration" is `settings.json` and "customization" is TypeScript.

What this book will cover

Everything. The stack from bottom to top, the decisions that made each layer look the way it does, and — where I can find it — the intuition behind choices that might not be obvious from the code alone.

Chapter 2 opens the lid: the shape of the monorepo itself, how the workspace is wired, and why the build script is a shell `cd` pipeline instead of Turbo or Nx.

The shape of the monorepo

Most modern TypeScript monorepos reach for Turbo or Nx on day one, pull in a task graph, and from then on nobody in the team ever runs a build without asking the graph to figure out what's dirty. pi-mono does not do this.

Here is the root `package.json` build script, in full, unedited:

```
"build": "cd packages/tui && npm run build && cd ../ai && npm run build && cd ../agent && npm run build && cd ../coding-agent && npm run build && cd ../mom && npm run build && cd ../web-ui && npm run build && cd ../pods && npm run build"
```

It is a `cd` pipeline. It is exactly as long as it looks, and the order is hand-maintained. This is the first thing that tells you something about the codebase: the author thinks the set of packages is small enough to enumerate in a shell one-liner, and the build dependency graph is stable enough not to need tooling to model it.

npm workspaces, nothing else

The workspace setup lives in the root `package.json`:

```
"workspaces": [
  "packages/*",
  "packages/web-ui/example",
  "packages/coding-agent/examples/extensions/with-deps",
  "packages/coding-agent/examples/extensions/custom-provider-anthropic",
  "packages/coding-agent/examples/extensions/custom-provider-gitlab-duo",
  "packages/coding-agent/examples/extensions/custom-provider-qwen-cli"
]
```

`packages/*` picks up the seven production packages. The explicit entries underneath pick up the handful of nested example extensions so that symlinked dev dependencies resolve locally. That's the whole monorepo.

No `turbo.json`. No `nx.json`. No Bazel, no Pants, no lerna. The root `package.json` and npm's built-in workspace resolver are the entire tooling story.

The compiler is `tsgo`, not `tsc`

Look at the root `devDependencies`:

```
"@typescript/native-preview": "7.0.0-dev.20260120.1"
```

That's `tsgo`, the TypeScript team's native port of the compiler (written in Go). The root `check` script:

```
"check": "biome check --write --error-on-warnings . && tsgo --noEmit && npm run  
check:browser-smoke && cd packages/web-ui && npm run check"
```

`tsgo --noEmit` replaces the old `tsc --noEmit`. The motivation is pure speed — `tsgo` is an order of magnitude faster on repos this size. The note in the root README nudges everyone to run `npm run build` first so `.d.ts` files exist for cross-package type checking: the `web-ui` package still uses `tsc` because it depends on the output of the other packages.

Biome is the only linter / formatter

`biome.json` is 28 lines. No ESLint, no Prettier, no `.editorconfig` fights. The `npm run check` command lints, formats (with `--write`), and type-checks in a single command. That's it.

The payoff is that contributors and AI agents alike have exactly one command to run after touching code. AGENTS.md is explicit:

After code changes (not documentation changes): `npm run check` (get full output, no tail). Fix all errors, warnings, and infos before committing.

Lockstep versioning

The harder tooling decision: **every package always has the same version number**. When you release, you release all seven packages at once.

From AGENTS.md:

Lockstep versioning: All packages always share the same version number. Every release updates all packages together.

The release script (`scripts/release.mjs`) enforces this. The root `package.json` has `version:patch`, `version:minor`, `version:major` scripts that run `npm version -ws --no-git-tag-version` (bump all workspace packages), then `node scripts/sync-versions.js` (which walks dependents and updates the `dependencies:` pins to match), then wipes `node_modules` and reinstalls to resolve the new pins.

The tradeoff: if you only changed `pi-ai`, you still cut a new `pi-tui`. Package consumers who want only `pi-ai` pay for releases they don't care about.

The benefit: **the version number is a commit label**. `pi-ai@0.30.4`, `pi-coding-agent@0.30.4`, `pi-tui@0.30.4` are guaranteed to have been released from the same commit on main. No need to maintain a compatibility matrix ("this version of coding-agent wants these versions of the libs"), because there is no matrix — every row is identical.

For a product that's tightly integrated end-to-end, this is the right call. For a library suite with independent consumers, it would be frustrating. `pi-mono` is firmly the first kind of repo, even though some of its pieces are good

libraries.

No major versions

Also in AGENTS.md:

Version semantics (no major releases):

- `patch` : Bug fixes and new features
- `minor` : API breaking changes

There's no `version:major` release step. Breaking changes ship as minor bumps. This is legitimate semver abuse, but deliberate. The projects haven't hit "1.0" and the author seems uninterested in the ritual. You're pinned to exact versions (`^0.30.4` under semver rules pins you to `0.30.x` for additive changes, but `0.x.y → 0.(x+1).0` is a free-for-all in semver). The convention here is `minor = breaking`, and the `CHANGELOG.md` files in each package tell you what actually changed.

The test runners

- **pi-ai** uses Vitest. Lots of cross-provider tests that need real API keys to run (skipped gracefully when keys aren't present).
- **pi-tui** uses Node's built-in test runner.
- **pi-coding-agent** has its own `test/suite/` with a `harness.ts` and a "faux provider" that synthesizes deterministic LLM responses so suites run without network or paid tokens. More on that in chapter 19.

The root `./test.sh` is a 35-line shell script that skips provider-dependent test files when no API keys are set. No fancy orchestration — `grep` for skip markers, run what's left.

`.pi/` — the agent's own config directory

The repo root has a `.pi/` directory. It is both:

1. **The project-scoped config the `pi` CLI itself uses** when you run it in this repo. Extensions, skills, settings for the repo's own agent.
2. **The published dataset** of coding sessions, under `.pi/hf-sessions/`. (These are gitignored locally but published separately to Hugging Face.)

Seeing `.pi/` at the root of the repo that *defines* pi is a funny kind of self-reference: pi-mono uses pi to work on pi-mono.

What the shape tells you

The tooling choices — npm workspaces instead of a task runner, tsgo instead of tsc, biome instead of ESLint+Prettier, lockstep versioning, shell-script build ordering — all point the same direction: **the repo is sized to fit in one head**. The author is not trying to scale to a hundred contributors. He is trying to keep friction low for one person (plus a few agents) hacking across seven packages.

That choice has knock-on effects everywhere in the code. Interfaces are small. Abstractions are thin. Names are short. When something looks under-engineered, it's usually because being easy to read was worth more than being generalized.

Chapter 3 pulls that thread: the explicit and implicit rules the codebase follows about what *not* to build.

Design philosophy

Nobody in pi-mono wrote a manifesto file, and if they had I wouldn't trust it. The rules a codebase actually follows are visible in what it does and what it refuses to do. Reading it for a while, here are the ones I see.

Rule 1 — Start with the minimum. Add nothing until forced.

The obvious shape of a modern coding agent goes something like:

- a plan / think / act loop
- a planner subagent, a critic subagent, a tool router subagent
- an MCP client with twenty connected servers
- a memory / RAG layer
- a vector store
- a framework of "chains" or "graphs"

pi-coding-agent has none of those. It has:

- one LLM stream
- four tools: `read`, `write`, `edit`, `bash`
- a queue for user messages while the model is working
- a JSONL file that records what happened

Everything else is an extension that ships *separately* from the minimum. Plan mode, MCP servers, memory files, subagents — all optional. The AGENTS.md at the root is explicit about this:

Aggressively extensible, minimal core, no MCP/sub-agents/plan-mode built-in — these are implemented as extensions/packages instead.

The intuition: if you bake in the wrong abstraction early, it becomes a tax on every feature forever. Start with nothing; let patterns emerge from use.

Rule 2 — One way to do things.

For most decisions, pi-mono picks one answer and commits. A sampling:

- **Config.** Global at `~/.pi/settings.json`, project-scoped at `./.pi/settings.json`. Deep merge, project overrides global. That's the whole system. There's no `.piconfig.yml`, no `pi.config.ts`, no environment-variable fanout with a precedence table.

- **Persistence.** Sessions are JSONL files. Not SQLite, not a git-backed store, not a cloud API. JSONL appends line by line, is trivially diffable, and can be opened in `less`.
- **State.** The `Agent` class in `pi-agent-core` has one mutable state object. Not an event store, not Redux, not an immutable tree. Setters that copy on assign; getters that return the live object.
- **Tool schemas.** TypeBox. Not Zod, not JSON Schema hand-written, not a custom DSL.
- **Linters / formatters.** Biome.
- **Compiler.** `tsgo`.

The cost: less flexibility when someone wants option B. The benefit: the code never goes off on a tangent to accommodate the third way.

Rule 3 — Put the honest tradeoffs in-code.

Read the comments in pi-tui's rendering path (`packages/tui/src/tui.ts` around line 1098):

Only render changed lines (firstChanged to lastChanged), not all lines to end. This reduces flicker when only a single line changes (e.g., spinner animation).

This comment isn't describing the code. It's describing *why the code is shaped like that* — naming the enemy (flicker) and the scenario (spinners) that forced the design. The codebase has a lot of these. They are the shortest form of design documentation available and they travel with the code.

Contrast with:

```
// Increment i by one
i++;
```

You find zero of those. The default is to write no comment. When a comment does appear it exists because someone decided a future reader would be confused without it.

Rule 4 — No `any`. Actually, no.

From AGENTS.md:

No `any` types unless absolutely necessary

This is enforced by biome, but the rule goes further: **don't invent types, read `node_modules` for the real ones.** Also from AGENTS.md:

Check `node_modules` for external API type definitions instead of guessing

And:

NEVER remove or downgrade code to fix type errors from outdated dependencies; upgrade the dependency instead

That last one is interesting. A common move in a hurry is "cast to `any` to quiet the error and move on." pi-mono explicitly rejects this. If the types say the code is wrong, the code is wrong or the types need updating. The shortcut is banned.

Rule 5 — Keybindings are configuration, not constants.

Also from AGENTS.md:

Never hardcode key checks with, eg. `matchesKey(keyData, "ctrl+x")`. All keybindings must be configurable.

Every key is looked up by a *name* in a `KeybindingsManager`. The `Editor` component never says "if this is Ctrl+A, move to line start." It says "if this matches `tui.editor.lineStart`, move to line start." The mapping from names to physical keys lives in a config the user can override.

For a TUI-heavy product, this is a rule you only get right if you write it down. Otherwise `ctrl+x` checks metastasize everywhere the first time someone wants a Vim mode.

Rule 6 — Don't preserve backward compatibility unless asked.

Also from AGENTS.md:

Do not preserve backward compatibility unless the user explicitly asks for it.

And the flip side:

Always ask before removing functionality or code that appears to be intentional.

Those sit together on purpose. "Don't write a compat shim I didn't ask for" is paired with "but don't delete something that looks load-bearing without checking." The code avoids museum debt (old exports kept around because of a hypothetical consumer) without letting refactors quietly break real users.

Rule 7 — Extensions are sandboxless, because trust is the model.

From the coding-agent research (chapter 12 will go deeper):

No sandbox: Extensions run in the same Node.js process as the agent. Trust model: Extensions are assumed trusted; they have full access.

Modern secure-coding orthodoxy says extensions should be sandboxed, run in a V8 isolate, capability-restricted. pi-mono just... doesn't. An extension is a TypeScript file you wrote or installed, running in the same process as `pi`.

The tradeoff is explicit: **pi treats "you ran this extension" as equivalent to "you ran this code."** No pretense otherwise. If that worries you, don't install extensions you don't trust. The benefit is that extensions have the full power of Node.js and the full pi API without adapter layers.

For a personal tool with a hacker audience, this is defensible. For a hosted product it would be negligent. pi is the first kind.

Rule 8 — Cross-provider fidelity is worth real engineering.

`packages/ai/src/providers/transform-messages.ts` is 172 lines of nothing-but-compatibility work. It rewrites tool call IDs, drops redacted-thinking blocks when moving to a model that can't replay them, strips Google's `thoughtSignature`, injects synthetic "No result provided" tool results to satisfy APIs that reject orphaned tool calls, and filters out errored assistant messages that would be replayed as invalid.

That file exists because the author took a conversation started against Claude, fed it to GPT, watched it explode, and kept the scar tissue in code. It makes "hand a conversation to another model and keep going" a real feature instead of an aspiration.

The test file that exercises it, `cross-provider-handoff.test.ts`, takes real contexts from 20+ providers and asks each target model to consume all of them. Failures dump payloads to `/tmp/pi-handoff-*.json` for offline triage.

Rule 9 — The event stream is the contract.

Neither `pi-ai` nor `pi-agent-core` throws on errors that are part of the model's normal output. Context overflow, rate limits, auth failures, aborts — all arrive as an event in the stream `( { type: "error", reason: "aborted", error: message } )` with a proper `AssistantMessage` attached.

Exceptions are reserved for things the caller *really* did wrong (calling `prompt()` while the previous `prompt()` is still running, for example).

This inversion — "soft" failures as data, "hard" failures as exceptions — is unusual enough to call out. The payoff is that UIs streaming the event stream need exactly one error path: render the error event. No try/catch wrapping every stream consumption.

Rule 10 — Intuitions, not abstractions.

A final pattern worth naming: pi-mono prefers to encode intuitions as small, concrete helpers rather than general abstractions.

Example: the "synthetic tool result" injection. Instead of a `MessageValidator` abstraction with rules and a pipeline, you get 40 lines of procedural code that says "if the last message had tool calls and the next one doesn't answer them, push a synthetic error result."

Example: "partial JSON parsing during tool call deltas." Instead of a streaming parser class with events, you get a function that tries `JSON.parse` and falls back to the `partial-json` npm package.

The rule seems to be: **write the procedure. If the same procedure shows up three times, pull it into a function. Don't invent a framework for it.**

These aren't in any style guide. They are the rules I see the code following. Violate them and your PR will look out of place; match them and the codebase feels consistent across ten-thousand lines and seven packages written over years.

Part II is four chapters deep on the four foundation packages. We start at the bottom of the stack: pi-ai.

PART II

The four foundation packages

pi-ai — one API, many providers

`pi-ai` is the bottom of the stack. Its job is exactly one thing: give you a single streaming function that talks to any of a large number of LLM providers, with enough fidelity that a conversation started against Claude can be handed off to GPT without the receiving model rejecting it.

The public surface is small:

```
import { stream, getModel } from "@mariozechner/pi-ai";

const model = getModel("anthropic", "claude-sonnet-4-6");

for await (const event of stream({ model, context })) {
  // text · tool_call · thinking · usage · done | error
}
```

Two functions, a few types. Everything else is implementation.

The `Context` type

`packages/ai/src/types.ts:223-227`:

```
export interface Context {
  systemPrompt?: string;
  messages: Message[];
  tools?: Tool[];
}
```

Three fields. That's the conversation. `Context` is **intentionally mutable**: the canonical pattern is to push an assistant message onto `context.messages` when a turn completes, then push the next user message, then call `stream` again. You are not supposed to build a fresh `Context` per call.

The upstream tradeoff is REPL-shaped rather than request-shaped, which matches how the code is actually used. The downstream effect is that `messages` accumulates forever unless someone (chapter 11) compacts it.

`Message` is a discriminated union of `UserMessage` | `AssistantMessage` | `ToolResultMessage`. Each `AssistantMessage` carries `provider`, `api`, and `model` metadata (lines 193-195), so when you later feed the conversation to a different provider, the library knows where each turn originated and can decide whether a thinking block is safe to replay.

The provider registry is lazy

pi-ai supports 20+ providers. Each provider is a few hundred lines of TypeScript doing message-shape translation, streaming parsing, and error mapping. Loading all of them up-front would be expensive in memory and startup time, especially in browser or serverless targets.

The trick is in `packages/ai/src/providers/register-builtins.ts`. Every provider has a module-level promise that fires exactly once:

```
anthropicProviderModulePromise ||= import("./anthropic.js").then(...)
```

The `||=` assignment is the whole lazy-load mechanism. Call `streamAnthropic()`, the import fires, subsequent calls reuse the resolved promise. No import, no cost. The `pi-ai` root export registers lazy wrappers; the actual provider modules only get loaded when you actually stream through them.

Bedrock is the exception (lines 329-343): AWS SDK is Node-only and too heavy to load speculatively, so it's gated behind an explicit `setBedrockProviderModule()` call. Test environments inject a mock.

The event stream

When you call `stream()` you get back an `AssistantMessageEventStream`, an async iterator of discriminated events. `types.ts:237-249`:

event	payload highlights	meaning
<code>start</code>	<code>partial: AssistantMessage</code>	Stream opened; empty skeleton.
<code>text_start</code> / <code>text_delta</code> / <code>text_end</code>	index, delta, final text	Text block being written.
<code>thinking_start</code> / <code>thinking_delta</code> / <code>thinking_end</code>	same shape	Extended thinking (Anthropic / OpenAI).
<code>toolcall_start</code> / <code>toolcall_delta</code> / <code>toolcall_end</code>	partial JSON, final <code>ToolCall</code>	A tool call being assembled.
<code>usage</code>	<code>Usage</code>	Final accounting.
<code>done</code>	<code>reason, message</code>	Success; final <code>AssistantMessage</code> attached.
<code>error</code>	<code>reason, error: AssistantMessage</code>	Failure; <code>stopReason: "error" "aborted"</code> .

Two details worth knowing.

First, `toolcall_delta` emits partial JSON. A model streaming

`{"name": "read", "args": {"path": "/etc/"}` will deliver it a chunk at a time, and `parseStreamingJson()` in `packages/ai/src/utils/json-parse.ts` will return a best-effort parse at every step. Internally the fast path is `JSON.parse`; on failure it falls back to the `partial-json` npm package. Incomplete fields come back as `undefined`. This lets UIs show tool arguments populating live instead of waiting for the call to finish.

Second, **errors travel as events, not exceptions**. A rate limit, a context overflow, a user abort, a provider 500 — all arrive as `{ type: "error", reason, error: AssistantMessage }`. The attached message has `stopReason` set and `errorMessage` filled in. Consumers never need `try / catch` around `for await`.

Cross-provider message transforms

This is the file I think is the most impressive 172 lines in the repo.

`packages/ai/src/providers/transform-messages.ts` implements `transformMessages<TApi>()`, which takes messages produced against provider A and prepares them for provider B.

Five transforms matter:

1. **Tool call ID normalization** (lines 14, 76-81). OpenAI Responses API emits IDs with pipes and 450+ characters; Anthropic requires `^[a-zA-Z0-9_-]+$` up to 64 chars. A map tracks the original → normalized mapping so tool result references downstream get rewritten consistently.
2. **Thinking block handling** (lines 41-56). Three cases:
 - *Redacted thinking* (OpenAI encrypted reasoning payloads). Dropped when switching models; kept only for replay against the same model, because only that model can decrypt it.
 - *Empty thinking* blocks (some providers emit them). Stripped.
 - *Cross-model fallback*. A thinking block moving to a model that doesn't support thinking becomes a plain `text` block so nothing is silently lost.
3. **thoughtSignature stripping** (lines 71-73). Google-specific opaque context. Harmless there, rejected as "unknown field" everywhere else.
4. **Synthetic tool results** (lines 98-171). The critical invariant: if an assistant message has tool calls and the next message is not a matching tool result, inject a synthetic one:

```
result.push({
  role: "toolResult",
  toolCallId: tc.id,
  toolName: tc.name,
  content: [{ type: "text", text: "No result provided" }],
  isError: true,
  timestamp: Date.now(),
});
```

This prevents "orphaned tool call" errors that most APIs reject at validation.

5. **Error/aborted message filtering** (lines 126-134). Assistant messages with

`stopReason: "error" | "aborted"` are dropped when re-submitting. They represent incomplete turns; replaying them tends to produce errors like OpenAI's "reasoning without following item."

The test file that exercises this, `cross-provider-handoff.test.ts`, feeds real captured contexts from every provider to every target model. Failures dump payloads to `/tmp/pi-handoff-*.json`. This is how a feature earns the right to be called "cross-provider": by proving it against a matrix of actual conversations, not a hand-written fixture.

Tools

Tools are defined with `TypeBox` (`@sinclair/typebox`). A `Tool` is `{ name, description, parameters: TSchema }` and nothing else (lines 217-221). The `parameters` field is expected to hold the JSON Schema output of `TypeBox`, not the `TypeBox` descriptor.

Each provider converts the generic `Tool` into its native format. For Anthropic (`anthropic.ts:867-883`):

```
return tools.map(tool => ({
  name: isAuthToken ? toClaudeCodeName(tool.name) : tool.name,
  description: tool.description,
  input_schema: { type: "object", properties: ..., required: ... },
}));
```

For OpenAI (`openai-completions.ts:717-731`) it wraps in the `type: "function"` envelope and sets `strict: false` because many OpenAI-compatible providers choke on strict mode.

Argument validation happens in `packages/ai/src/utils/validation.ts`. AJV compiles the JSON Schema and validates (and coerces) the parsed arguments. If code generation isn't available (strict CSP, some browser extensions, lines 14-25), AJV is disabled and validation is skipped. That's a reasonable tradeoff: some environments just can't.

The Claude Code tool name round-trip

One of the delightful hacks in this codebase. When you authenticate to Anthropic via the "Claude Pro/Max" OAuth flow, the API expects tool names in Claude Code's canonical PascalCase: `Read`, `Write`, `Bash`, `TodoWrite`. But pi calls its tools `read`, `write`, `bash`, `todowrite`.

`anthropic.ts:64-101` handles this. On outbound, `toClaudeCodeName()` maps lowercase tool names to their PascalCase equivalents. On inbound, `fromClaudeCodeName()` maps them back. The mapping is a case-insensitive exact-match lookup against the tools array — so if your tool is registered as `todowrite`, the API sees `TodoWrite`, and the returned tool call shows up as `todowrite` again on your end. Round-trips cleanly; test in `anthropic-tool-name-normalization.test.ts`.

This is a good example of a rule-9 design: a named concept ("ClaudeCode tool names") with a tiny, procedural implementation.

Token accounting

`Usage` carries input, output, cacheRead, cacheWrite, total, plus a `cost` object priced in USD. Cost is computed from `model.cost` metadata (auto-generated in `models.generated.ts`) and the usage numbers, multiplied and divided by a million. That's literally the whole calculation (`packages/ai/src/models.ts:39-46`).

Cache tokens are reported where providers support them:

- Anthropic reports them natively on every streamed usage block.
- OpenAI requires `stream_options: { include_usage: true }` to surface `prompt_tokens_details.cached_tokens`.
- Google reports them unevenly, occasionally omitting cache granularity.

If a provider doesn't report cache tokens, they stay zero; total is computed without them.

Context overflow detection

`packages/ai/src/utils/overflow.ts:28-131`. `isContextOverflow()` uses eighteen regex patterns across fifteen providers to detect overflow from error messages — `/prompt is too long/i`, `/request too large/i`, `/exceeds the context window/i`, and others, with exclusion patterns to rule out rate limits that happen to mention tokens.

It also catches **silent overflow** (lines 123-128): a `stopReason: "stop"` response where `usage.input + usage.cacheRead > contextWindow`. Some providers (looking at you, z.ai) accept overflowing requests and return successfully with partial output. The library treats that as an overflow so callers can react the same way.

Overflow detection is a primitive for the coding agent's compaction logic; we'll come back to it in chapter 11.

Unicode surrogate sanitization

One of the small, unpleasant realities of working with LLM output: models occasionally produce unpaired UTF-16 surrogates, especially on emoji that got chopped mid-chunk. JSON APIs reject these. Pi-ai strips them everywhere it serializes: `sanitizeSurrogates()` in `types.ts:29` and its friend in `packages/ai/src/utils/sanitize-unicode.ts:21-25`.

The regex:

```
/[\uD800-\uDBFF](?![\uDC00-\uDFFF])|(?<![\uD800-\uDBFF])[\uDC00-\uDFFF]/g
```

A high surrogate without a trailing low surrogate, or a low surrogate without a leading high, gets removed. Valid emoji (properly paired surrogates) pass through. There is no graceful path that repairs broken pairs — they are stripped and the data moves on.

OAuth

Five providers support OAuth in pi-ai:

1. Anthropic (Claude Pro / Max)
2. GitHub Copilot (device flow)
3. Google Gemini CLI (gcloud refresh)
4. Google Antigravity (Cloud Code Assist)
5. OpenAI Codex (ChatGPT OAuth)

The registry is in `packages/ai/src/utils/oauth/index.ts:35-48`. `getOAuthApiKey()` at line 119 is the whole refresh loop:

```
if (Date.now() >= creds.expires) {  
  creds = await provider.refreshToken(creds);  
}  
return { newCredentials: creds, apiKey: provider.getApiKey(creds) };
```

Callers must persist the returned `newCredentials`, because tokens rotate. The persistence story lives one layer up, in `pi-coding-agent`'s `AuthStorage` (chapter 13).

The largest files

A rough map of where complexity concentrates, for when you go reading the source yourself:

file	~lines	why
<code>providers/google-gemini-cli.ts</code>	987	OAuth refresh + reasoning block interleaving
<code>providers/openai-codex-responses.ts</code>	970	Codex-specific reasoning adapter
<code>providers/anthropic.ts</code>	905	Cache control, thinking, OAuth tool-name round-trip
<code>providers/openai-completions.ts</code>	881	Covers the largest fleet of compatible providers
<code>providers/amazon-bedrock.ts</code>	807	AWS Converse API + SDK integration
<code>providers/register-builtins.ts</code>	433	Lazy-load orchestration
<code>providers/transform-messages.ts</code>	172	Cross-provider glue

The providers are chunky; everything else is small. That distribution tells you where the domain difficulty actually lives.

In one paragraph

pi-ai is a thin, lazy, event-stream-driven abstraction over a large number of LLM providers. The two things that earn it its keep are (a) cross-provider message fidelity — the transform layer — and (b) treating errors as events, so

upstream consumers stream everything uniformly. Everything else — OAuth, tools, tokens, cache — is competent, unremarkable plumbing that stays out of the way.

Next chapter: `pi-agent-core`, which takes `stream()` and builds the smallest honest agent runtime around it.

pi-agent-core — an agent is just a loop

"Agent framework" is a phrase that, over the last two years, has come to mean something between 1,000 and 30,000 lines of code. `pi-agent-core` is about 650 lines across `agent.ts`, `agent-loop.ts`, `types.ts`, and `proxy.ts`. You can read the whole thing in an afternoon and there will be nothing left to understand.

The central claim of the package is that **an agent is a loop that (1) calls a model, (2) runs whatever tools the model asked for, and (3) goes back to step 1 until the model stops asking**. Everything else — permissions, parallelism, streaming events, error recovery — is annotations on that loop.

Two public entry points

```
import { Agent, agentLoop, agentLoopContinue } from "@mariozechner/pi-agent-core";
```

`agentLoop(config, messages)` takes new user messages, pushes them into the conversation, and runs the loop until the model stops. It returns an `EventStream` immediately — the caller starts consuming; the loop runs on another tick and feeds events in.

`agentLoopContinue(config)` is almost the same but for **resuming without a new user message**. Its precondition is that the last message in context is *not* an `assistant` message (`agent-loop.ts:70-76`) — either it's a `toolResult`, or a `user` message that hasn't been answered. This is what you use to retry after a tool error, or after the LLM stream aborted mid-turn. No dummy "continue" message required.

The `Agent` class at `agent.ts:157-539` is a higher-level wrapper holding mutable session state. You use it when you want conveniences like "current streaming message," a pending tool call tracker, steering and follow-up queues, or an `activeRun` concept that prevents double-prompt calls. Under the hood it calls `agentLoop` / `agentLoopContinue`.

The state machine

Internally the `Agent` holds a single `MutableAgentState` (see the setter pattern at `agent.ts:74-85`, which copies arrays on assign to prevent external mutation):

```
systemPrompt    // string
model           // pi-ai Model
thinkingLevel   // "off" | "minimal" | "low" | "medium" | "high" | "xhigh"
tools           // AgentTool[]
messages        // AgentMessage[]
isStreaming     // boolean
streamingMessage? // partial AssistantMessage while streaming
pendingToolCalls // Set<string> – tool call IDs currently executing
errorMessage?   // last error text from a failed turn
```

Lifecycle is driven by `runWithLifecycle()` at lines 434-456:

```
idle
├─ prompt() or continue() called
│   ├─ activeRun = { promise, resolve, abortController }
│   ├─ isStreaming = true
│   ├─ errorMessage = undefined
│   └─ executor runs runAgentLoop or runAgentLoopContinue
└─ finally
    ├─ isStreaming = false
    ├─ streamingMessage = undefined
    ├─ pendingToolCalls cleared
    └─ activeRun = undefined
→ idle
```

Concurrent calls to `prompt()` or `continue()` throw immediately (lines 313-316, 324-326). The agent is **single-threaded by design**. If you want parallelism, you run multiple agents.

Steering and follow-up queues

The most unusual thing in `Agent` is that it has two separate message queues with independent drain modes. From `agent.ts:55-143`:

```
class PendingMessageQueue {
  mode: "all" | "one-at-a-time";
  messages: AgentMessage[];
  drain(): AgentMessage[] {
    if (mode === "all") { const m = [...messages]; messages = []; return m; }
    return messages.length ? [messages.shift()!] : [];
  }
}
```

Two instances, each controlled by its own setter:

- **Steering queue.** Messages the user sends *while the agent is already streaming or running tools*. Drained after each `turn_end` (`agent-loop.ts:216`) — so a steering message interrupts the next model call, but not mid-stream.
- **Follow-up queue.** Messages the user sent after the agent would have otherwise stopped (no more tool calls, no more steering). Drained only when the inner loop is about to exit (lines 169-228) — so follow-ups reactivate a finished agent instead of interrupting a running one.

Why two? Because "what should a rapid click do?" has different answers in different UIs.

- A chat UI where each Enter should interrupt the current response: `steeringMode: "all"`, drain all queued messages into one big turn.
- A careful UI where each message should be answered individually, even if the user typed five in a row: `"one-at-a-time"`, queue up, drain one per turn.

For the coding agent, the default is `"one-at-a-time"` for both. For a chat app you might flip to `"all"`. The choice is per-instance.

Tools: `AgentTool` extends `Tool`

`types.ts:292-307` :

```
interface AgentTool<TParameters extends TSchema, TDetails = any> extends Tool<TParameters>
{
  label: string; // human-readable
  prepareArguments?: (args: unknown) => Static<TParameters>;
  execute: (
    toolCallId: string,
    params: Static<TParameters>,
    signal?: AbortSignal,
    onUpdate?: AgentToolUpdateCallback<TDetails>,
  ) => Promise<AgentToolResult<TDetails>>;
}
```

The `Tool` base (from pi-ai) is the schema. `AgentTool` adds `label` (for UI rendering), an optional `prepareArguments` hook to massage raw arguments before validation, and the `execute` function itself. A tool reports progress via `onUpdate` (for long-running operations that want to stream status to the UI).

Tools throw on error — they do not encode errors as part of their result content. The loop catches the throw, wraps the message, and emits an error-flavored tool result.

The hooks: `beforeToolCall`, `afterToolCall`

This is how permission systems, sandbox gates, and extensions intercept behavior. Both are async, both receive an `AbortSignal`.

`beforeToolCall` receives the tool call and its *already-validated* arguments:

```
type BeforeToolCallContext = {
  assistantMessage: AssistantMessage;
  toolCall: AgentToolCall;
  args: unknown; // validated against tool.parameters
  context: AgentContext;
};

type BeforeToolCallResult = {
  block?: boolean; // true → do not execute
  reason?: string; // error message if blocked
};
```

It's a permission gate. It cannot modify arguments; if you want to change them, do it in `prepareArguments` on the tool itself.

`afterToolCall` receives the real result and can override fields:

```
type AfterToolCallResult = {
  content?: (TextContent | ImageContent)[]; // replace
  details?: unknown;                        // replace
  isError?: boolean;                       // replace
};
```

Omitted fields keep their original values. There's no deep merge — each field is either replaced entirely or left alone.

The hooks are how the coding agent's interactive mode prompts you to approve a `bash` call before it runs, and how audit extensions log the exact result the model saw.

Sequential vs parallel tool execution

The loop has two implementations behind one flag (`config.toolExecution`):

```
type ToolExecutionMode = "sequential" | "parallel";
```

Default: `"parallel"`. A single assistant turn that produces three tool calls runs them concurrently by default.

The interesting detail is that **even in parallel mode, results are emitted in original source order**. Lines 390-437 of `agent-loop.ts`: prepare all calls sequentially (catching validation errors), then launch runnable ones concurrently as a Promise array, then await them one by one in source order — so tool result events arrive in the order they appeared in the assistant message, not the order they finished. The downstream UI and the LLM both see a consistent ordering.

The event timeline

Here's the exact order of events for a prompt that produces one assistant message with one tool call, followed by a second assistant response to the tool result:


```

prompt("hello")
agent_start
turn_start
message_start { userMessage }
message_end   { userMessage }
message_start { assistantMessage }
message_update { assistantMessageEvent: "text_delta" }
message_update { assistantMessageEvent: "toolcall_start" }
message_update { assistantMessageEvent: "toolcall_delta" × N }
message_end   { assistantMessage }
tool_execution_start { toolCallId, toolName, args }
tool_execution_update { partialResult } (optional)
tool_execution_end   { result, isError }
message_start { toolResultMessage }
message_end   { toolResultMessage }
turn_end     { message, toolResults }
turn_start   (second turn)
message_start { assistantMessage }
message_end   { assistantMessage }
turn_end     { message, toolResults: [] }
agent_end    { messages }

```

Every event is a plain JSON object. `message_update` delegates the inner `assistantMessageEvent` to pi-ai's event types directly, so the whole pipeline is one stream of discriminated unions from the provider API up to the UI.

Backpressure — the two-level model

A subtlety that caught me on the first pass.

The **low-level** `agentLoop()` (`agent-loop.ts:276-330`) emits events into an `EventStream` and **does not await subscribers**. If a consumer is slow, events queue; the producer keeps going. This is deliberate — the loop should never block on a UI that isn't keeping up.

The **high-level** `Agent` class (`agent.ts:535-537`) wraps this by awaiting *all* subscriber promises in `processEvents()` before advancing. So when you use `Agent`, a slow subscriber does apply backpressure: subsequent events pile up *before* being emitted, not after.

Both behaviors are valid. If you want the no-backpressure model, consume `agentLoop()` directly. If you want the "wait until everyone has caught up" model, use the `Agent` class. The code makes the choice explicit and the README calls it out at line 452.

Error recovery

Three kinds of failure:

1. **A tool throws.** Caught at `agent-loop.ts:524-559`. The error is wrapped in a `createErrorToolResult()` and emitted as a tool result with `isError: true`. The loop continues — the LLM sees the error and can decide to retry, explain, or ask the user.

2. **The LLM stream errors or aborts.** Detected at line 194 of the loop:

`message.stopReason === "error" || "aborted"`. The agent emits `turn_end`, then `agent_end`, and exits. The final assistant message carries `errorMessage` so the UI can display it.

3. **Tool argument JSON parse failure.** Caught in the try/catch around `prepareToolCall()` (lines 515-521). Emitted as an error tool result; the loop continues.

The `Agent` class wraps all of this in an additional top-level catch (lines 459-474) that synthesizes an `assistant` message with `stopReason: "error"` if something really unexpected bubbled up. `errorMessage` gets set on the agent state so UIs can display it between turns.

What agent-core does *not* do

It does not detect or handle context overflow. Overflow detection lives in pi-ai (`isContextOverflow()`, chapter 4) and recovery (compaction, retry) lives in pi-coding-agent (chapter 11). Agent-core just emits the error and moves on.

It does not persist anything. The `messages` array stays in memory. Saving to disk, branching, forking — all the coding-agent's problem.

It does not know about TUIs, files, users. It's a runtime. The `StreamFn` abstraction means you can swap pi-ai for a mock, or for the `streamProxy()` below.

`streamProxy` — the remote-agent escape hatch

`proxy.ts` (341 lines) implements a `streamProxy()` function that replaces pi-ai's `stream()` but routes the call through a backend HTTP endpoint. The use case is browser apps that can't hold API keys directly but want to use the full `Agent` locally.

The protocol is a bandwidth-optimized version of the event stream. The server emits `ProxyAssistantMessageEvent` objects (lines 36-57) with the `partial: AssistantMessage` field *stripped* — the client reconstructs the partial locally from deltas in `processProxyEvent()` (lines 211-340). For `text_delta`, it appends to `partial.content[index].text`. For `toolcall_delta`, it appends to a JSON buffer and parses incrementally.

End result: the `Agent` class doesn't know it's running over a proxy. You pass a different `streamFn` at construction time and everything else works identically.

What it looks like from the outside

```

import { Agent } from "@mariozechner/pi-agent-core";
import { stream, getModel } from "@mariozechner/pi-ai";

const agent = new Agent({
  streamFn: stream,
  model: getModel("anthropic", "claude-sonnet-4-6"),
  systemPrompt: "You are helpful.",
  tools: [
    {
      name: "calculator",
      description: "Do arithmetic",
      label: "Calculator",
      parameters: T.Object({ expr: T.String() }),
      execute: async (_id, { expr }) => ({
        content: [{ type: "text", text: String(eval(expr)) }],
      }),
    },
  ],
});

agent.on("message_end", (ev) => console.log(ev.message));
await agent.prompt("what is 7 * 6?");

```

That is the full surface area of the package for most uses. Model, system prompt, tools, prompt. It doesn't invent anything that isn't there.

Next: pi-tui. Same package, different concern — how do you render any of this in a terminal without flicker?

pi-tui — a terminal that doesn't flicker

Most terminal apps flicker. You notice it when a spinner updates and the whole screen winks. You notice it when `htop` redraws or a loader animates and your eyes track a faint gray strobe. The terminal wasn't designed to be an animation target; it was designed to be a line printer that occasionally moves its cursor.

`pi-tui` gets around this with three ideas — string-level line diffing, synchronized output, and rendering overlays pre-diff — and a handful of careful details underneath.

The render contract

A component is anything that implements:

```
interface Component {  
  render(width: number): string[];  
  handleInput?(data: string): void;  
  wantsKeyRelease?: boolean;  
  invalidate(): void;  
}
```

`render(width)` returns an array of strings — one per line, each fitting within `width` visible columns. That's it. No JSX tree, no virtual DOM, no reconciliation. A component's job is to produce lines.

The `TUI` class (`packages/tui/src/tui.ts:217`) is the root container. It owns the terminal, the `previousLines` array from the last frame, the overlay stack, and the render throttle. Every frame is a pass through the component tree producing an array of strings, then a diff against the last frame's strings, then the minimum ANSI required to bring the terminal up to date.

The enforcement detail at `tui.ts:1105` is the rule that keeps this simple:

```
if (visibleWidth(line) > width) {  
  // crash with a debug dump  
}
```

A component that returns an over-wide line is considered buggy and the TUI hard-fails with a log dump. This means the diff engine can assume every line fits exactly; no re-flow, no clipping, no surprise.

The diff

At `tui.ts:985-996`:

```
for (let i = 0; i < maxLines; i++) {
  const oldLine = i < this.previousLines.length ? this.previousLines[i] : "";
  const newLine = i < newLines.length ? newLines[i] : "";
  if (oldLine !== newLine) {
    if (firstChanged === -1) firstChanged = i;
    lastChanged = i;
  }
}
```

Two indices: first and last line that changed. Everything between them gets re-emitted (with an EL — "erase line," `\x1b[2K` — before each); everything outside is untouched.

It's line-level, not cell-level, and it's exact string compare. The content of each line is a single string including ANSI color codes; if the codes change, the string changes, the line redraws. No state machine, no ANSI-aware diff. The author took the tradeoff that occasionally you will redraw a 500-char line because one color code at the end changed, and that's fine.

Three strategies layer on top:

1. **Full repaint with clear** (`tui.ts:918-942` , `962`). Triggered on width change, on first render, or on Termux-style height changes (the Android terminal that lies about its size). Emits `CSI 2J` (clear screen) + `CSI H` (home) + `CSI 3J` (clear scrollbar), then writes all lines.
2. **Differential update** (`tui.ts:1069-1155`). Normal path. Moves the cursor to the first changed line, `\x1b[2K` before each new line, writes content.
3. **Overlay compositing** (`tui.ts:734-794` , `compositeOverlays`). Base content is rendered, then overlays at their positions are composited *into* the line array, then the composed array is diffed. This means an overlay appearing or changing only redraws the base lines it covers — not the whole screen.

The flicker rationale, in code

The comment at `tui.ts:1098` is as close to a design statement as this package has:

*Only render changed lines (firstChanged to lastChanged), not all lines to end. **This reduces flicker when only a single line changes (e.g., spinner animation).***

A spinner updating every 80ms is the canonical case. If you redraw every line, the entire viewport flickers; if you redraw just the spinner's line, only that line flickers — and it flickers no more than any single character changing would. Apply the same principle to streaming LLM text (one line changing per token burst, scrollbar stable) and you get an agent UI that feels alive instead of nauseating.

CSI 2026 — synchronized output

The second piece. Modern terminals support CSI 2026 (also written `?2026h` / `?2026l`), a pair of control sequences that tell the terminal "don't present intermediate state while I'm writing." The buffer between them is rendered atomically.

pi-tui wraps every render in a pair:

```
\x1b[?2026h
...cursor moves, erase lines, content...
\x1b[?2026l
```

Emitted at `tui.ts:920` (`fullRender`) and `1071` (`diffRender`). Terminals that don't understand the sequence silently ignore it; output still works, you just lose atomicity. No capability detection, no fallback logic. The author bet on modern terminals and took the simpler code path.

Between line-level diffing and synchronized output, the result is that a pi-tui screen updates the way a web page does: the screen never shows a half-drawn state. For streaming UIs, this is the difference between "looks like an agent is thinking" and "looks like my terminal is broken."

Overlays

`OverlayOptions` (`tui.ts:119-155`) supports anchor-based positioning (8 cardinals + center), explicit row/col in pixels or percents, margins, width and max height in either unit, a `visible(termWidth, termHeight) => boolean` callback for responsive hide/show, and a `nonCapturing` flag that determines whether the overlay captures keyboard focus.

Overlays live in an `overlayStack[]` array with `focusOrder` counters. When compositing (lines 743-744), they are sorted by `focusOrder` ascending and drawn in that order — later overlays on top. Focus cycles through the topmost *visible* overlay; hidden overlays don't steal the keyboard.

The compositing-before-diffing choice is what makes overlays basically free. A modal popping up only redraws the rectangle the modal covers. A modal updating only redraws the modal. The rest of the screen stays exactly as it was.

Input

Keyboard input comes from stdin as raw bytes, and decoding it is a swamp. pi-tui handles it in two layers.

`stdin-buffer.ts:29-126` is the low-level reassembler. It recognizes CSI (`ESC[`), OSC (`ESC[`), DCS (`ESC P`), APC (`ESC _`), and old-style mouse sequences (`ESC[M` + 3 bytes). It only emits `data` events when a complete sequence is in hand, so consumers never see half an escape.

`keys.ts` is the semantic decoder. It understands the Kitty keyboard protocol and falls back to `modifyOtherKeys` mode 2 for xterm-family terminals (tmux, Konsole, older xterm) that don't support Kitty.

Detection logic, `terminal.ts:184-194`:

1. On startup, emit `\x1b[?u` (query Kitty protocol).
2. Wait 150ms. If the terminal replies with `\x1b[?<flags>u`, set `_kittyProtocolActive = true`, enable the protocol with `\x1b[>7u` (flags: disambiguate escapes, report event types, report base layout key).

3. If no reply within 150ms, enable `\x1b[>4;2m` (xterm modifyOtherKeys mode 2) as a fallback so you can at least distinguish `Ctrl+Shift+T` from `T`.

The binary state flag `_kittyProtocolActive` decides which of two key parsers runs in `keys.ts`.

Keybindings are resolved through a `KeybindingsManager` (`keybindings.ts:155-201`) that maps *names* (like `tui.editor.cursorUp`) to arrays of key IDs, any of which can match. First match wins; there is no priority or propagation story. A keybinding is owned by exactly one handler, and swapping themes or user configs swaps which key triggers which name.

This is the mechanical enforcement of "keybindings are configuration" from chapter 3.

Bracketed paste

The terminal sends `\x1b[200~...content...\x1b[201~` around pasted text. Components buffer between the markers (`input.ts:47-82`, `editor.ts:252-258`) and treat the whole buffered chunk as a single logical "paste" event.

Large pastes (>4 KB) are tracked in a paste registry with numeric IDs. In the editor, the paste gets collapsed to a placeholder like `[paste #3 (1200 lines)]`, which stands in for the content and can be selected and deleted as a single unit. The real text is kept in the registry so commit / render operations can expand it. This is how the editor stays responsive when someone pastes a 10 MB file.

Images

On terminals that support it, pi-tui renders real images (not ASCII art). Detection is env-var based (`terminal-image.ts:40-71`):

- `KITTY_WINDOW_ID`, `TERM_PROGRAM=kitty`, Ghostty, WezTerm → Kitty graphics protocol.
- `ITERM_SESSION_ID`, `TERM_PROGRAM=iterm.app` → iTerm2 inline images.
- Everything else (VS Code terminal, Alacritty, generic) → null capability; the `Image` component renders a text fallback.

No Sixel support. The author bet on Kitty + iTerm2 covering most of the audience and left Sixel for later.

Kitty encoding (`encodeKitty`, 106-147) splits base64 data into 4 KB chunks and sends them as a sequence of APC escapes. Each chunk carries `a=T` (transmit), `f=100` (24-bit PNG), `q=2` (no display on transmit), plus optional `c=<cols>`, `r=<rows>`, `i=<imageId>` for reuse across frames.

Cell dimensions (so you know how many rows a 500px image is) come from a CSI 16t query at startup; results cached.

The Editor

`components/editor.ts:217-296` is a full multi-line editor with:

- `state.lines[]` — the buffer
- A sticky column for vertical movement (`line 272`): when you move up through a short line, the next down-arrow returns you to your original column if the new line is long enough
- An `UndoStack<EditorState>` of 100 snapshots (line 348)
- An Emacs-style kill ring with concatenation of consecutive kills (`lastAction` tracking)
- A paste markers table for inline collapse of big pastes
- Pluggable autocomplete via `AutocompleteProvider` , debounced 20 ms (line 215)
- A separate command-history up/down behavior

It does *not* have built-in syntax highlighting — that's delegated to an `EditorTheme` callback so you can bring your own highlighter.

Cursor positioning trick

There's a neat bit at `tui.ts:867-886` . Components don't directly tell the TUI where to park the hardware cursor after rendering. Instead, the component embeds a zero-width APC marker (`\x1b_pi:c\x07`) at the cursor's logical position in the output string. The TUI scans rendered lines for this `CURSOR_MARKER` , computes the column by measuring visible width up to the marker, strips the marker from the line, and emits `\x1b[<row>;<col>G` to place the terminal cursor there.

The benefit: component authors write normal strings. The cursor is a piece of their output, not a separate piece of bookkeeping. This matters when a component's content moves — if the cursor travels with the content, the TUI doesn't need extra logic to track "the cursor belongs to component X and component X shifted down by 2 lines."

East Asian width

The `get-east-asian-width` dependency plugs into `graphemeWidth` in `utils.ts:156-194` . Every visible-width calculation in pi-tui flows through this function, so double-width characters (CJK, full-width punctuation, some emoji) are accounted for consistently — in `visibleWidth` , `truncateToWidth` , `sliceByColumn` , word wrapping, and the over-wide-line enforcement at `tui.ts:1105` .

One special case: regional indicator symbols (U+1F1E6 .. U+1F1FF, the components that pair up into flag emoji) are hand-forced to width 2 because terminals are inconsistent about them and the default would cause "auto-wrap drift" during streaming.

Performance numbers

The comments reveal the measured targets:

- `MIN_RENDER_INTERVAL_MS = 16` (tui.ts:230): ~60 FPS cap
- 150 ms: Kitty protocol detection timeout
- 20 ms: autocomplete debounce
- 100: max undo snapshots

- 512: width cache size

All tiny numbers. The package doesn't benchmark itself; the design bet is "terminal I/O is the bottleneck, keep the render loop fast and cheap and the rest follows." Line-level diffing + synchronized output

- per-component caching achieves that without intricate data structures.

Windows

Node on Windows has a standing "raw stdin doesn't work" problem for anything involving modifier keys.

`terminal.ts:111, 202-223` uses `koffi` to load the native DLL and set `ENABLE_VIRTUAL_TERMINAL_INPUT` on the console mode so Shift+Tab arrives as `ESC[Z` instead of a raw tab byte, and so on. Without this, Kitty protocol detection wouldn't even get to fire.

This is the kind of plumbing you only know about if you've tried to ship a TUI on Windows. It's in the repo because the author apparently tried.

In one paragraph

pi-tui is a line-level, diff-based, synchronized-output-native TUI engine with overlays composited before the diff, a strict width contract that keeps the diff simple, input handling that understands Kitty and falls back gracefully, and just enough Windows plumbing to actually work there. It is not a React-for-terminals abstraction. Components return strings; the engine puts those strings on the screen.

Next chapter is where pi-ai, pi-agent-core, and pi-tui meet the user: pi-coding-agent.

pi-coding-agent — the thing you actually run

This is the biggest package in the repo and the one that has your name on it when you install the CLI as `pi`. It is also the one that ties everything together: `pi-ai` for streams, `pi-agent-core` for the loop, `pi-tui` for the terminal, and a lot of glue.

The entry point

`src/cli.ts:8-17`:

```
process.title = "pi";
// disable some warnings, set undici proxy
await main(process.argv.slice(2));
```

That's it. `main()` lives in `src/main.ts` and is the actual orchestrator. The chain is:

```
cli.ts (set title, delegate)
└─ main.ts
    │ parseArgs          (cli/args.ts)
    │ resolveAppMode     (interactive | print | json | rpc)
    │ createSessionManager
    │ createAgentSessionRuntime
    └─ runMode           (InteractiveMode / runPrintMode / runRpcMode)
```

parseArgs

`cli/args.ts` handles a lot more than you'd think from the name. The parser:

- Supports `@file` syntax for loading a prompt from a file (line 163).
- Validates thinking levels (lines 118-127).
- Accumulates **unknown flags** into a `Map<string, string>` (line 47-48) — so extensions can define their own flags and read them from this map.
- Collects diagnostics as it goes (line 49), which the final output surfaces after startup completes.

Flags like `--model`, `--provider`, `--thinking` support compound syntax.

`--model claude-sonnet:high` is parsed as model `claude-sonnet` with thinking `high`. The split happens later in `model-resolver.ts:180-200`.

Mode selection

`main.ts:97-108`'s `resolveAppMode()`:

- `--mode rpc` → **rpc mode** (JSON-RPC on stdin/stdout)
- `--mode json` → **json mode** (streams all events as JSONL on stdout)
- `--print`, or stdin is a pipe → **print mode** (one-shot, final text)
- Otherwise, stdin is a TTY → **interactive mode** (TUI)

The guard at `main.ts:451`: if mode isn't interactive and stdin isn't a TTY, `takeOverStdout()` redirects `console.log` / `console.error` to internal buffers. Extensions and tools that log to stdout would otherwise corrupt the RPC protocol or the single-shot text output. `restoreStdout()` at line 723 puts it back before exit.

AgentSession and AgentSessionRuntime

There are two classes here and the distinction matters.

`AgentSession` (`src/core/agent-session.ts`) owns the **conversation state**: the messages, the steering and follow-up queues, the compaction state, the current tool registry. It inherits from the `Agent` class in `pi-agent-core` and adds coding-agent-specific behavior.

`AgentSessionRuntime` (`src/core/agent-session-runtime.ts:54-126`) owns the **services bound to a working directory**: the session manager writing to a JSONL file, the auth storage, the extension runner, the tools, the cwd itself. It's the thing that gets torn down when you switch sessions.

Separating them means `/switch` and `/fork` have a clean lifecycle. `switchSession()` (line 128) tears down the runtime's services, closes the old session, and mounts a new one. Extensions get `session_before_switch` and `session_before_fork` events before the teardown so they can persist anything they need to (lines 83-111).

The session file — JSONL all the way down

`src/core/session-manager.ts`. A session file is a sequence of line-delimited JSON entries. The first line is the session header:

```
{ "type": "session", "version": 3, "id": "abc123", "cwd": "/home/user/proj", "timestamp":
  "..."} }
```

Then messages, thinking-level changes, compactions, and custom entries, each with an `id` and `parentId`:

```
{ "type": "message", "id": "msg1", "parentId": null, "message": { ... } }
{ "type": "message", "id": "msg2", "parentId": "msg1", "message": { ... } }
{ "type": "thinking_level_change", "id": "tlc1", "parentId": "msg2", "thinkingLevel":
  "high" }
{ "type": "compaction", "id": "cmp1", "parentId": "msg2", "summary": "...",
  "firstKeptEntryId": "msg2" }
{ "type": "custom_message", "id": "cm1", "parentId": "cmp1", "customType": "...",
  "content": "..."} }
```

The file is **append-only**. A branch is just multiple entries that share the same `parentId`. There are no "delete" entries; rolling back means picking a different leaf. Chapter 10 pulls this apart further.

Concurrency. The session manager and the auth storage both use `proper-lockfile` (`src/core/auth-storage.ts:62-87`, `src/core/settings-manager.ts:152-177`), with 10 retries and 20 ms delay. This matters because a user might run two `pi` instances in the same project — lockfile acquisition prevents file corruption from concurrent writes.

Compaction

`src/core/compaction/compaction.ts` provides the pure summarization logic; `AgentSession` orchestrates when to call it.

Trigger:

```
After each turn:
  tokens = calculateContextTokens(lastUsage) + estimate(trailingMessages)
  if tokens > contextWindow - reserveTokens:
    compact()
```

Estimation (`compaction.ts:135-213`): real usage when the model reported it, falling back to a `chars / 4` heuristic for messages without usage and a fixed `4800 tokens per image` for image content.

The summarization prompt (in `compaction/utils.ts`) is straightforward and was tuned for the "technical decisions must survive" use case. It's called via `completeSimple()`, not streaming, because we need to know the final token count before continuing.

The tricky part is the **overflow recovery flag** (`_overflowRecoveryAttempted`). If context overflow happens mid-turn, the session attempts one compaction and retries. If the retry also overflows, the agent emits an error instead of spinning forever. This caps bad loops at two attempts.

Compaction entries land as children of the current leaf. Subsequent messages are children of the compaction. When reconstructing the conversation for the next LLM call, the tree walker emits: (1) the compaction's summary text as a system or user message, (2) all entries from `firstKeptEntryId` up to the compaction point, (3) all entries after the compaction. No history is rewritten — compaction is just another entry in the append-only log. Chapter 11 unpacks this in detail.

The four built-in tools

`src/core/tools/`. Each one is an `AgentTool` with a `parameters` schema, a `label`, and an `execute`.

`bash.ts`

Spawns the child with `detached: true` so it gets its own process group (POSIX). Stdio is `["ignore", "pipe", "pipe"]` — stdin ignored to prevent interactive commands from hanging, stdout/stderr captured.

Output streams incrementally via `onData` callback (so the TUI can show output as it arrives) *and* is tee'd to a temp file, which lets the tool return the full output even if its streamed snippet gets truncated for the LLM.

Timeout: if exceeded, `killProcessTree()` walks the group and kills everyone. No SIGTERM-then-SIGKILL escalation — one kill. If a child ignores signals and orphaned its grandchildren, the grandchildren survive. In practice this is rare enough to be acceptable.

`read.ts`

Default limits: 5000 lines or 100 KB, whichever hits first (line 15). `offset` is 1-indexed (to match `grep` / editor numbering); `limit` caps line count. A special case at line 103: if the very first line exceeds the byte limit, it truncates at a line boundary mid-line so we don't feed the LLM a half-character.

Image handling: `read` on a known image extension returns structured content with `[{ type: "text", text: "...", }, { type: "image", data, mimeType }]`. Large images are optionally resized via `resizeImage()` (lines 160-162) to a 2000×2000 bounding box so the LLM doesn't pay for a 4K photo.

`write.ts`

Always overwrites. No append mode. No atomic-write guarantee on all filesystems — it's `writeFileSync`, which the author considered sufficient. Tools that need append semantics should use `edit`.

`edit.ts`

The model passes an array of `{ oldText, newText }`. Each `oldText` must be an exact substring match, unique in the file, and not overlapping with any other edit in the same call. The whole batch is applied against the original file, not incrementally, so edits never see each other's changes.

Input is normalized to LF before matching; the original line endings (CRLF or LF) are restored after. BOM, if present, is stripped before matching and re-added at the end. Returns a diff string for display.

This is deliberately unforgiving. No fuzzy matching, no whitespace-insensitive mode. If the LLM's `oldText` doesn't match exactly, the edit fails, the LLM sees the failure, it tries again. The strictness is what prevents "the file looks fine but I silently skipped an edit" bugs.

The helpers

`grep.ts` wraps `ripgrep`. `find.ts` uses `rg --files` (so `.gitignore` is respected). `ls.ts` is a `ls -la`-style listing.

Extensions

Anything the core doesn't do — MCP, plan mode, sub-agents, memory, additional tools, UI widgets, slash commands — lives here.

`src/core/extensions/loader.ts` uses `@mariozechner/jiti` (a TypeScript runtime loader) to `import()` an extension file at runtime. It sets up import aliases (lines 77-84) so extensions can `import { Something } from "@mariozechner/pi-coding-agent"` and have the symbols resolve to the bundled versions rather than a second copy.

In Bun binary mode (when `pi` was compiled to a single executable), the loader uses `virtualModules` (lines 43-50) instead of file resolution. Everything else is the same.

Discovery. The `ResourceLoader` scans `~/.pi/extensions/` (global) and `./.pi/extensions/` (project-local). Explicit `--extension=path` flags add more. `--no-extensions` disables auto-discovery entirely.

API surface (lines 161-200 of the loader):

method	purpose
<code>on(event, handler)</code>	Subscribe to <code>message_start</code> , <code>tool_execute_end</code> , <code>compaction_end</code> , etc.
<code>registerTool(tool)</code>	Declare an LLM-callable tool.
<code>registerCommand(name, opts)</code>	Register a <code>/command</code> .
<code>registerShortcut(keyId, opts)</code>	Bind a keyboard shortcut in interactive mode.
<code>registerProvider(name, cfg)</code>	Register a custom LLM provider (for when you want to point at something pi-ai doesn't ship).
<code>registerMessageRenderer(type, fn)</code>	Custom rendering for custom message types.
<code>registerSlashCommand(name, desc, fn)</code>	Sugar for simple slash commands.

No sandbox. Trust, not isolation. Chapter 12 goes deeper.

Lifecycle. Extensions load *before* `bindCore()` (lines 134-151 in `loader.ts`), so their registration calls happen against stub action methods that throw "not initialized" if invoked during load. After bind, the runner replaces the stubs with live implementations. This prevents extensions from accidentally calling session actions before the session is ready.

Skills vs prompt templates vs extensions

Three similar-sounding things. The differences matter.

Skills (`src/core/skills.ts:74-81`): directory-based, stateless prompt templates with frontmatter metadata. Invoked by the user as `/skill:name`. Contents are plain Markdown. Skills cannot execute code. Example: a skill for "code review" is Markdown describing how to review code; invoking it inserts that Markdown as a user message.

Prompt templates: fragments injected into the system prompt at startup. Like skills but for always-on context, not on-demand invocation.

Extensions: executable TypeScript. Everything the other two can't do.

The rule: "text" → skill, "system-prompt additive" → prompt template, "code" → extension. No confusion between them because they literally don't share code paths.

Settings

Global at `~/.pi/settings.json`, project at `./.pi/settings.json`. Deep merge, project overrides global. Loading and merging in `src/core/settings-manager.ts:102-130`. Nested objects merge recursively; arrays and primitives are replaced entirely.

Typical settings:

```
{
  compaction?: { enabled?: boolean; reserveTokens?: number };
  branchSummary?: { reserveTokens?: number };
  theme?: string;
  extensions?: string[];      // extra extension paths
  sessionDir?: string;       // custom session storage
  enabledModels?: string[];   // for Ctrl+P cycling
}
```

Initialization at `main.ts:486`:

```
const startupSettingsManager = SettingsManager.create(cwd, agentDir);
```

Settings writes also use `proper-lockfile` to serialize concurrent `pi` instances in the same project.

Three modes, three chapters

- **Interactive** (`src/modes/interactive/`) — the TUI you usually see. Theming, keybindings, input editor, message renderers, extension UI callbacks.
- **Print** (`src/modes/print-mode.ts`) — single-shot. `pi "fix the readme"` runs one turn and exits. `--mode json` spits events on stdout.
- **RPC** (`src/modes/rpc/`) — stdin/stdout JSON-RPC for IDE extensions and pi-mom.

Chapter 14 takes them one at a time.

A bunch of small, clever things

Scattered through the package, some intuitions worth calling out now because the chapters they belong to are about something else:

Model auto-fallback. `src/core/model-resolver.ts:151-165`. If you ask for a model that's not in the registry, the resolver builds a synthetic `Model` object with the provider's defaults so the CLI doesn't hard-fail on

an unknown model name. A new Anthropic model the registry hasn't heard of yet still works; the accounting just may not be accurate until you upgrade.

Dual thinking level sources. `--thinking high` takes precedence over `--model claude-sonnet:low`. Useful for CLI scripts: you can pin the thinking level regardless of what model you ended up resolving (agent-session.ts:320-345, main.ts:350-351).

Session CWD validation. If you `/resume` a session whose `cwd` no longer exists, the runtime refuses to mount it and prompts you with a fallback cwd (main.ts:387-419). A subtle case — the session metadata points at `/Users/you/old-project`, which you deleted three months ago, and the agent can't possibly do file operations there.

File mutation queue. The edit tool imports a `withFileMutationQueue()` helper that batches file operations so two tools in the same turn don't race on overlapping ranges. The coding agent runs tools in parallel by default; this queue is the reason that's safe for file edits.

Session file recovery. If the session file is empty or corrupted on load, `setSessionFile()` truncates and starts fresh with a valid header (lines 700-706). It's better to lose an unreadable session than to keep erroring on every load.

In one paragraph

pi-coding-agent is the product. A CLI argument parser, an orchestrator of session/runtime lifecycles, four built-in tools, a sandboxless extension system loaded via `jiti`, a compaction engine wired to pi-ai's overflow detection, a lockfile-guarded JSONL persistence layer, and three I/O modes over the same `AgentSession` core. The bulk of the code is in the modes and the tools; the core coordination is smaller than you'd guess.

Part III moves from "what each package is" to "how the patterns work across the stack." Starting with how events actually flow.

PART III

Cross-cutting design

Event streams and backpressure

A recurring pattern across pi-mono: **the primary interface between layers is an async iterator of discriminated-union events**. pi-ai's `stream()` returns one. pi-agent-core's `agentLoop()` returns one. The coding-agent's RPC mode emits one as JSON lines on stdout. The web-ui renders one into HTML.

This is the stack's spine. Picking it apart is the point of this chapter.

The shape of an event

Every event in the codebase has a `type` discriminator and a payload that depends on the type. Example from pi-ai:

```
type AssistantMessageEvent =
  | { type: "start"; partial: AssistantMessage }
  | { type: "text_delta"; contentIndex: number; delta: string }
  | { type: "text_end"; contentIndex: number; content: string }
  | { type: "thinking_delta"; ... }
  | { type: "toolcall_start"; ... }
  | { type: "toolcall_delta"; ... }
  | { type: "toolcall_end"; contentIndex: number; toolCall: ToolCall }
  | { type: "done"; reason: string; message: AssistantMessage }
  | { type: "error"; reason: "error" | "aborted"; error: AssistantMessage };
```

pi-agent-core wraps pi-ai's events into a higher-level event type that nests the inner `assistantMessageEvent`, adds `tool_execution_*`, and frames everything in `agent_start` / `turn_start` / `turn_end` / `agent_end`.

The coding-agent's RPC mode serializes those as JSON lines. Each line is a single event. Clients parse and render.

Why events, not callbacks

Two reasons.

First, **composability by transformation**. An event stream can be transformed — filtered, mapped, merged — without changing the consumer or producer. The cross-provider transform layer in chapter 4 is literally a function `(events, context) => events`. The RPC mode is `events.map(JSON.stringify)`. The web-ui is `events → DOM updates`. Callbacks would require reshaping every consumer.

Second, **no implicit state in the consumer**. An `async for await` loop over events is the simplest control flow for "do something until this stream ends." There is no handler registration to track, no dispose method to remember, no event emitter cleanup.

Soft errors, hard errors

Chapter 3 mentioned this as rule 9 and it's worth restating in detail because it's counterintuitive:

Errors travel as events, not exceptions.

Context overflow, rate limits, provider 500s, auth failures, user aborts — all arrive as

`{ type: "error", reason, error: AssistantMessage }`. The attached `AssistantMessage` has `stopReason: "error" | "aborted"` and a filled-in `errorMessage`.

A consumer's `for await` loop does not need `try / catch`. The last event before the loop ends tells you how it ended:

```
for await (const ev of stream({ model, context })) {
  switch (ev.type) {
    case "text_delta": appendText(ev.delta); break;
    case "toolcall_end": showToolCall(ev.toolCall); break;
    case "done": markSuccess(); break;
    case "error": showError(ev.error.errorMessage); break;
  }
}
```

Exceptions *are* used. They are reserved for caller misuse — calling `prompt()` on an already-running agent, passing an invalid model, feeding `agentLoopContinue` a context whose last message is an assistant message. Those genuinely are bugs on the caller side, and throwing is the right response.

The split is simple: **"the model or provider did something you need to react to"** is data. **"You used the API wrong"** is an exception.

The EventStream class

`packages/ai/src/utils/event-stream.ts:4-88` implements a queue-and-promise hybrid that every layer uses:

```
EventStream<T, R>
  queue: T[] // buffered events
  waiting: Array<(t: IteratorResult<T>) => void> // blocked consumers
  finalResult: Promise<R> // resolves when stream ends
```

`push(event)` either pops a waiter and fires it or appends to the queue. `next()` either pops from the queue or appends a waiter. When `complete(result)` runs, remaining waiters get `{ done: true }`, and `finalResult` resolves.

The two-level model: every layer has its own `EventStream`, and higher layers consume lower layers' streams and push into their own. That pattern is what lets you stack pi-ai underneath pi-agent-core underneath pi-coding-agent without any layer needing to know where its events ultimately go.

Backpressure: the two paths through agent-core

Recall from chapter 5 that `pi-agent-core` has two ways to consume:

```
// Low-level – no backpressure
for await (const ev of agentLoop(config, messages)) { ... }

// High-level – backpressure
const agent = new Agent(config);
agent.on("message_end", async (ev) => { await somethingSlow(); });
await agent.prompt("hi");
```

In the low-level path, `agentLoop()` pushes events into its `EventStream` and keeps going. Consumer is slow → queue grows.

In the high-level path, the `Agent` class's `processEvents()` (`agent.ts:535-537`) **awaits every subscriber's promise** before advancing. Consumer is slow → producer waits.

Which is right depends on what you're building.

- **The coding-agent's RPC mode** uses the high-level path with a subscriber that does a synchronous ``stdout.write(JSON.stringify(ev)`
 - `"\n")``. Since the subscriber is fast, the two paths are indistinguishable, but having the backpressure available means that if the consumer process is slow to read stdout, the agent pauses rather than unbounded-buffering.
- **pi-mom's Slack handler** uses the high-level path with subscribers that make `chat.postMessage` HTTP calls to Slack. Slack is slow (sometimes 500 ms per message). Backpressure means the agent doesn't get far ahead of what's displayed. Chapter 15 covers the trick pi-mom uses to avoid this serializing every thing.
- **The web-ui** uses pi-ai directly (no agent wrapper) and consumes events in a plain loop. DOM updates are fast enough that it doesn't matter; when they aren't, the author would need to pull events off the stream into a render queue manually.

Making the backpressure choice per-layer instead of one-size-fits-all is unusual and, as far as I can tell, intentional.

Abort signals everywhere

Every function that emits events also accepts an `AbortSignal`. `stream(model, context, { signal })`. `agentLoop(config, messages, signal)`. `agent.prompt("...", { signal })`. Chains all the way down.

When the signal fires:

1. The provider's HTTP client gets the abort and tears down the connection.
2. The pi-ai stream emits `{ type: "error", reason: "aborted", error: message }` where `message.stopReason === "aborted"`.

3. Agent-core's loop sees `stopReason` is `"aborted"` (line 194 of `agent-loop.ts`) and exits cleanly via `turn_end` → `agent_end`.
4. Any running tool receives the signal and can cancel itself (every tool's `execute` signature takes a `signal?`).
5. The `Agent` class reports `isStreaming = false` and clears `pendingToolCalls`.

All of this is one clean path through the code because the abort is a property of the event stream's signal, not a side-channel.

Where events come from in each layer



Each hop is a plain function-to-async-iterator conversion. There is no framework, no dependency injection, no event bus. The shape of the interface — "give me an async iterator of discriminated events" — stays the same from bottom to top.

Rendering an event stream in a TUI

Because pi-tui is diff-based, a streaming LLM response is implemented as: a component whose `render()` returns a growing string. Each `text_delta` appends to its internal state and calls `invalidate()`. The TUI's render throttle (16 ms, 60 fps) coalesces multi-token bursts into one render. The diff detects that only the streaming component's lines changed and redraws only those.

A `toolcall_delta` with partial JSON can similarly be rendered live. The code shows `Reading /etc/nginx` before the path is even complete, updating in real time as the model fills in arguments. Chapter 9 pulls this apart.

In one paragraph

Events are the spine of pi-mono. Every layer emits a stream of discriminated union events; errors travel as events except for genuine caller mistakes. The two-level backpressure model (`agentLoop` = no backpressure, `Agent` = await all subscribers) matches the two canonical consumers — fast loggers and slow UIs. `AbortSignal` threads through every call site so cancellation is a property of the stream, not a side channel. The whole system works because the interface between layers never changes.

Next: what a tool call actually looks like as it travels that spine.

Tool calling end-to-end

A single tool call touches every layer of pi-mono. Chapter 8 covered the event spine; this one walks one call from user input to file on disk and back, annotating the interesting bits.

The scenario

The user runs `pi "read packages/ai/src/types.ts and count the exported types"`. The agent decides to call the `read` tool with path `packages/ai/src/types.ts`.

Step 1 — user input arrives

In interactive mode, the `Editor` component collects the prompt. Pressing Enter fires `AgentSession.prompt(text)`. The session constructs a `UserMessage` with the text and any attachments, appends it to `context.messages`, persists it to the JSONL file, and calls `agentLoop()`.

In print mode the same thing happens without the editor; the argv string becomes the prompt.

In RPC mode, a `{ "type": "prompt", "message": "..."}` line arrives on stdin and is dispatched to `session.prompt()`.

Step 2 — the stream opens

`pi-agent-core` emits `agent_start`, then `turn_start`, then `message_start` for the user message, then calls `stream()` on pi-ai.

pi-ai formats the context for Anthropic (in this example), including the tool definitions:

```
"tools": [
  { "name": "read", "description": "...", "input_schema": { "type": "object",
"properties": { "path": ..., "offset": ..., "limit": ... }, "required": ["path"] } },
  { "name": "write", ... },
  { "name": "edit", ... },
  { "name": "bash", ... }
]
```

The HTTP POST fires. The SSE stream starts arriving.

Step 3 — the tool call streams in

Claude decides to call `read`. Events arrive chunk by chunk:


```

start
  partial = empty AssistantMessage

text_delta "I'll read that file for you."
  (displayed to user)
text_end
  (content block 0 closed)

toolcall_start
  contentIndex = 1
  partial.content[1] = { type: "tool_call", id: "toolu_abc", name: "read", args: undefined
}

toolcall_delta
  partial.JSON so far: `{"path":"packa`
  parseStreamingJson → { path: "packa" }

toolcall_delta
  `{"path":"packages/ai/src/types.ts"}`
  parseStreamingJson → { path: "packages/ai/src/types.ts" }

toolcall_end
  toolCall = { id: "toolu_abc", name: "read", args: {"path":"packages/ai/src/types.ts"} }

```

`parseStreamingJson` (`packages/ai/src/utils/json-parse.ts:10-28`) is the hero of this section. Fast path `JSON.parse`, fallback to the `partial-json` package. Incomplete strings become `undefined`; partial objects become real objects with the parsed subset of keys. The TUI can therefore display "Reading `packa` ..." and watch it fill in live.

Step 4 — validation

`pi-agent-core` consumes the `pi-ai` events and emits its own. When `toolcall_end` arrives it triggers `prepareToolCall()` at `agent-loop.ts:458-470`:

1. Look up the tool by name (`read`).
2. If the tool has `prepareArguments`, call it to massage the raw args. (`read` doesn't.)
3. Call `validateToolCall()` from `pi-ai`, which compiles the JSON schema with AJV and validates `{ path: "packages/..." }`.
4. If validation fails, emit an error tool result with the AJV error message and move on. The LLM sees the error and can retry.

Validation succeeded. Arguments are now typed as `Static<typeof ReadSchema>` — we have `{ path: string, offset?: number, limit?: number }`.

Step 5 — the before hook

`pi-agent-core`'s `beforeToolCall` hook fires (`agent-loop.ts:491-507`).

In interactive mode with approval required, this is where the TUI opens an overlay asking "Read `packages/ai/src/types.ts` ? [y/N/a]" and waits for a keypress. If you press `n`, the hook returns `{ block: true, reason: "user denied" }` and the tool is never executed. The LLM sees an error tool result; the agent continues.

If you press `y`, the hook returns `undefined` (or `{ block: false }`) and execution continues.

In RPC mode, the before hook dispatches an `extension_ui_request` event to the client, which handles approval however it wants. In print mode, the default config often has auto-approval turned on for read-only tools.

Step 6 — execution

`tool_execution_start` is emitted. The tool's `execute()` runs with the abort signal and an optional `onUpdate` callback.

`read.ts` :

```
execute: async (toolCallId, { path, offset, limit }, signal, onUpdate) => {
  const absolutePath = resolveReadPath(path, cwd);
  await checkSignal(signal);
  const content = await readFileSafely(absolutePath, { offset, limit, signal });
  return { content: [{ type: "text", text: content.text }], details: content.meta };
}
```

Read does:

1. Resolve `path` against `cwd` (absolute paths are absolute; relative are relative to the session's `cwd`, which might differ from `process.cwd()` after a `/switch`).
2. Check it's not binary, not huge (truncate at 100 KB / 5000 lines).
3. If it's an image, return `content: [{ type: "text", text: "..."}, { type: "image", data: base64, mimeType }]`.
4. Return the content.

On error (permission, missing file, binary) `execute` throws. The loop catches and wraps in `createErrorToolResult()`.

Step 7 — the after hook and tool result message

`tool_execution_end` fires with `{ result, isError }`.

The `afterToolCall` hook runs (`agent-loop.ts:573-592`). Extensions can override `content`, `details`, or `isError`. Omitted fields are untouched. A common use: a logging extension that writes the result to a file and returns `{}`, letting the LLM see the unmodified result.

pi-agent-core then synthesizes a `ToolResultMessage` :

```
{
  role: "toolResult",
  toolCallId: "toolu_abc",
  toolName: "read",
  content: [{ type: "text", text: "... file contents ..." }],
  isError: false,
  timestamp: Date.now(),
}
```

It pushes this onto `context.messages`. Emits `message_start` and `message_end` for the tool result. Emits `turn_end`.

Step 8 — second turn

The loop re-enters. Because the last message is `toolResult` (not `assistant`), it calls `stream()` again — this is the implicit "continue" that makes tool calls work. The tool result is now part of the context the LLM sees.

Claude generates a regular text response: "The file exports 17 types, including..." This streams in as `text_delta` events, gets packaged into an `AssistantMessage`, gets pushed to `context.messages`, emits `turn_end`, and because this turn has no tool calls, the loop exits with `agent_end`.

Step 9 — persistence

Everything that got pushed to `context.messages` also got appended to the JSONL session file as it happened. The session manager's `appendMessage()` is called from the agent's `message_end` subscriber. By the time `agent_end` fires, the file on disk is current.

If the process dies mid-stream, the file has every completed message up to the crash. Nothing was lost. The next launch of `pi --resume` can rebuild exactly the same state.

Parallel tool calls

If Claude had called `read` and `grep` in the same turn, the call changes slightly. Recall from chapter 5 that default tool execution is parallel.

```
toolcall_end for read
toolcall_end for grep
tool_execution_start { read }
tool_execution_start { grep }
tool_execution_end { read result }
tool_execution_end { grep result }
message_start { toolResultMessage: [read result, grep result] }
```

Both tools launch concurrently. Both results are collected. They get emitted in the order the model asked for them, not the order they finished. That ordering matters: LLMs see tool results in the same order as their tool call blocks

in the assistant message, and shuffling would confuse them.

When a tool wants to stream updates

The `onUpdate` parameter on `execute` is for long-running operations. `bash` uses it: as stdout and stderr arrive in chunks, it calls `onUpdate({ stdout, stderr, elapsed })` which flows out as a `tool_execution_update` event. The TUI shows this live.

```
tool_execution_start { bash "sleep 5; echo done" }
tool_execution_update { stdout: "" }
... (five seconds later)
tool_execution_update { stdout: "done\n" }
tool_execution_end { result, isError: false }
```

The LLM doesn't see intermediate updates — they're purely for UI. Only the final result goes in the tool result message.

Tool result content: text and images

A tool can return text or images or both. The `content` field of `AgentToolResult` is an array of `TextContent` | `ImageContent`. `read` on an image file returns both — a text description ("read PNG image, 800×600") and the actual base64 image data.

pi-ai's provider conversion layer handles packaging this into provider-native tool result formats. Anthropic and Gemini take multimodal tool results natively; for providers that don't, the image is dropped (with a log warning) and only the text survives.

Synthetic tool results

If you abort mid-stream — tool call emitted, but the process died before the tool ran — `context.messages` has an assistant message with a tool call and nothing follows it. Next time you prompt, the cross-provider transform layer (chapter 4) injects a synthetic error tool result:

```
{ "role": "toolResult", "toolCallId": "toolu_abc", "toolName": "read",
  "content": [{ "type": "text", "text": "No result provided" }], "isError": true }
```

The conversation stays valid. The LLM sees "oh, that tool didn't actually run" and proceeds from there.

In one paragraph

One tool call is: partial-JSON streamed arguments → AJV validation → before hook → `execute()` with signal and `onUpdate` → result → after hook → tool result message appended to context and JSONL → second turn re-

enters `stream()` with the tool result in scope. Parallel tool calls run concurrently but emit in source order. Every step is an event on the stream, every event gets persisted, the whole thing is resumable on crash.

Next chapter: the JSONL file where all this history lives, and the tree structure hiding inside it.

Sessions as JSONL trees

Every turn you have with `pi` gets appended to a session file. The file is JSONL — one JSON object per line. That's the whole storage engine. No database, no SQLite, no git-like object store.

But the file is not a flat log. Hidden in the `parentId` field is a tree, and that tree is the mechanism for `/fork`, `/tree`, compaction insertion, and crash recovery.

The on-disk format

A fresh session file, abridged:

```
{ "type": "session", "version": 3, "id": "abc", "cwd": "/home/user/proj", "timestamp": "..."}
{ "type": "message", "id": "m1", "parentId": null, "message": { "role": "user", "content":
  [ { "type": "text", "text": "hi" } ] } }
{ "type": "message", "id": "m2", "parentId": "m1", "message": { "role": "assistant", "content":
  [ ... ] } }
{ "type": "thinking_level_change", "id": "tlc1", "parentId": "m2", "thinkingLevel": "high" }
{ "type": "message", "id": "m3", "parentId": "tlc1", "message": { "role": "user", "content": [ ... ] } }
```

Five entry types today (`src/core/session-manager.ts`):

type	what it carries
<code>session</code>	Header: id, cwd, timestamp, version. First line only.
<code>message</code>	A user, assistant, or tool result message.
<code>thinking_level_change</code>	A record that the thinking level was changed between turns.
<code>compaction</code>	A summary entry that replaces prior history for the LLM.
<code>custom_message</code>	Extension-owned. Any shape, any meaning.

Every entry has `id` and `parentId`. `parentId: null` is reserved for the root; every other entry has a parent. The *last appended* entry is the current leaf — the tip of the "branch" you're working on.

Why JSONL

The author had options. SQLite is battle-tested. A git-backed store would naturally support branching. A proper event store would make replay trivial.

JSONL got picked because it is:

1. **Human-readable.** Open a session in `less`. Grep for a command you ran. Diff two sessions in `git diff --no-index`. This is genuinely useful for debugging agent behavior, and it's how the author publishes sessions as training data on Hugging Face.
2. **Append-only.** Writing is `fs.appendFile`. Never seek, never rewrite, never worry about torn writes. A crash mid-append leaves a truncated last line which a single pass discards.
3. **Lockable.** `proper-lockfile` with 10 retries over 200 ms (`auth-storage.ts:62-87`) is enough to serialize two `pi` instances in the same project.
4. **Infinitely streamable.** Reading is `fs.readFile` and split by `\n`. A 100,000-line session is still milliseconds to load.

The cost is that "loading" is linear in total entries and reconstructing an arbitrary subtree is $O(n)$. For sessions that stay under a few thousand entries (which is what a long day of work looks like), this is fine. Longer than that and compaction has probably already kicked in.

The tree inside

The flat file encodes a tree because multiple entries can share a `parentId`:



Here the user made a `/fork` at `m3`. Both `m4` and `m5` have `parentId: "m3"`. Two branches.

The reconstruction algorithm (`src/core/session-manager.ts:310-417`) is:

1. Build a flat `Map<id, entry>` index (lines 316-320).
2. Given a leaf id, walk `parentId` back to root, collecting entries in reverse (lines 341-347).
3. Reverse the collection to get chronological order.

That's the path from root to leaf — the actual conversation.

The leaf is whichever entry you're currently pointing at. Normally it's the *last appended* entry, since new work extends the current branch. After `/fork`, it's the entry you forked from (the new branch starts empty). After `/tree` → `select`, it's whichever entry you picked.

`/fork` in detail

The exposed semantics: "I want to go back to an earlier state and try something else without losing what I already did."

Under the hood (`src/core/session-manager.ts:1076-1100`):

1. Create a new session file.
2. Write a `session` header whose `parentSession` field points at the current session's id (genealogy tracking).
3. Copy the path-to-leaf from the parent session into the new one.
4. Return a handle to the new session; the current runtime `switchSession` es to it.

The original session file is untouched. Two separate files now exist. You can pick either, `/switch` between them, fork each independently.

It's cheap because copying a 500-entry session is copying 500 JSON lines. It scales because forking never rewrites history.

`/tree`

`/tree` opens a tree-browser overlay showing the current session's entries as a structured outline with branches. Up/Down navigates; Enter selects; Space expands.

When you select an entry:

- The runtime "rewinds" the agent's state to that point by rebuilding `context.messages` from root to the selected entry.
- The selected entry becomes the new leaf.
- The next thing you prompt will be a child of that entry.

This is how pi supports the workflow "that last tool call was wrong, let me steer the conversation differently from two steps back." Without `/tree`, the only way would be to undo by asking the model to undo, which rarely works well.

Compaction and the tree

From chapter 7, compaction inserts a `compaction` entry into the tree. A compaction entry has:

```
{
  type: "compaction",
  id: string,
  parentId: string,    // current leaf when compaction ran
  summary: string,     // LLM-generated summary of prior history
  firstKeptEntryId: string, // where "original context" resumes
  tokensBefore: number,
  details?: unknown,
  fromHook?: boolean,
}
```


When reconstructing the conversation for the next LLM call, the tree walker

(`src/core/session-manager.ts:385-408`) behaves specially when it encounters a compaction in the path:

1. Skip directly to the compaction entry.
2. Emit a pseudo-message containing `summary`.
3. Emit the original entries from `firstKeptEntryId` up through the compaction's parent.
4. Emit entries *after* the compaction normally.

In practice this means: "if you compacted 80 messages of history, the next LLM call sees the summary plus the last N kept messages plus whatever came after the compaction." The full history is still in the file — compaction never deletes — but the LLM-visible context is compact.

Chapter 11 goes deeper on the triggers and the prompt.

Custom messages

Extensions can write arbitrary `custom_message` entries. An extension that implements "pinned files" might insert a custom message for each pin event. An extension that implements a local memory file might insert a custom message each time it was updated.

When the tree is read back, custom messages can be rendered by the extension's own

`registerMessageRenderer`. When the LLM context is rebuilt, they can be converted to user or system messages by the extension's `convertToLlm` callback on the `AgentLoopConfig`.

The JSONL format intentionally doesn't forbid anything under `custom_message`. Extensions own their data; the core just persists whatever shape they give it.

Concurrency: two `pi` instances in the same project

The session manager and `AuthStorage` both use `proper-lockfile`. At `src/core/auth-storage.ts:62-87` you'll see:

```
const release = await lockfile.lock(path, {
  retries: { retries: 10, minTimeout: 20, maxTimeout: 200 },
});
try {
  // ... do the read+write
} finally {
  await release();
}
```

10 retries over 200 ms covers every realistic case of two instances fighting over the same file. If one is somehow hung with the lock forever, the lock file's stale-detection kicks in and a future instance takes over.

Note this doesn't serialize two instances against *different* session files in the same project. Two instances working on different sessions in the same project can run simultaneously with no conflict. It's only writes to the same file

that serialize.

Session file corruption

A truncated last line happens if the process dies mid-append. The loader handles it (`src/core/session-manager.ts:700-706`): if parsing any line throws, and the malformed line is the last one, it's dropped. If it's an earlier line, the session is considered broken.

There's a further belt-and-braces fallback: if `setSessionFile()` finds the file exists but has no valid `session` header, it truncates and starts fresh. The rationale: a session file with no header is effectively garbage; better to lose it than to error on every load forever.

Genealogy

Session headers carry:

- `id` — this session's id
- `parentSession` — the session this was forked from, if any
- `cwd` — the working directory this session was started in
- `timestamp` — when it was created

The genealogy chain is not currently exposed in the UI, but the data is there. You could walk `parentSession` back through a tree of forks and trace the full history of how a session came to be.

The parallel-write story you don't have to worry about

Two internal guarantees make concurrency basically not a thing for callers:

1. Within a single `pi` process, only one `Agent` runs at a time per session (chapter 5's "concurrent calls to `prompt()` throw").
2. Across processes, `proper-lockfile` serializes writes.

You cannot race tool results into the wrong session, you cannot get a half-written entry, you cannot get two instances writing interleaved entries. The system looks like a single-writer log from the outside, even though the file is accessed from multiple places (message append, thinking-level change, compaction, extension writes).

In one paragraph

A session is a JSONL file where each entry has an id and a parentId, encoding a tree that supports branching, forking, compaction-as-entry, and streaming append for crash safety. The file is human-readable, grep-able, trivially diffable, and locked by `proper-lockfile`. Every "state" the agent might need (rewind, fork, compact, resume, custom-extension-event) is just a different way to interpret the same append-only log. This is the single best design decision in the repo.

Next: compaction in detail, because the brief version in chapter 7 skipped the interesting parts.

Context compaction

LLMs have fixed context windows. Every token you stuff in costs money and takes time; past a certain point the model's quality starts to degrade; past the hard limit, the API errors out.

Compaction is pi's mechanism for dealing with this: when the context gets too big, summarize the old stuff into a shorter summary, keep recent messages intact, and continue the conversation from the summary. It is a small idea with a lot of sharp edges.

When compaction fires

Three triggers, in order of likelihood:

1. **Auto-compaction.** After each turn, the session checks context size and compacts if it's too close to the limit. Controlled by settings (`compaction.enabled` , `compaction.reserveTokens`).
2. **Manual / extension-triggered.** A user types `/compact` or an extension calls `session.compact()` . Forces one now regardless of size.
3. **Overflow recovery.** The API returned a context-overflow error mid-turn. Pi detects the overflow pattern (chapter 4's `isContextOverflow`), compacts, and retries. This is capped at one attempt to avoid infinite loops.

Counting tokens

The check is:

```
contextTokens = calculateContextTokens(lastAssistantUsage)
                + estimateTokens(messagesAfterLastUsage)
if contextTokens > model.contextWindow - reserveTokens:
    compact()
```

`calculateContextTokens` uses the real usage the provider reported on the last assistant message. Providers' `usage.input + usage.cacheRead` is the authoritative count of tokens that went into the last prompt.

But tokens have been added *since* that measurement — whatever user messages, tool results, and custom messages were appended after. `estimateTokens()` covers those with a heuristic:

- Text: `length / 4` (the commonly cited English estimate).
- Images: a flat 4800 tokens each (`compaction.ts:272`). Not precise; close enough to set the trigger reliably.
- Tool results: text estimate for text content, image estimate for images.

The heuristic is wrong in predictable directions — English-like text is roughly 4 chars/token; non-ASCII can be denser. It errs toward overestimating for safety; the worst case from being wrong is that compaction fires slightly

early, not that overflow happens.

The `reserveTokens` buffer

The trigger compares against `contextWindow - reserveTokens`, not `contextWindow` itself. Default `reserveTokens` is around 10,000 (the actual number depends on settings). The reason:

1. The next turn is going to add *more* context before the model responds — the user's next message, the model's response, any tool calls. We need headroom.
2. The usage report from the provider is on the *input* to the last turn. The assistant's own output (which becomes input to the next turn) isn't counted until we estimate it.
3. Providers sometimes lie or round. Leave a margin.

The summarization prompt

```
src/core/compaction/utils.ts:SUMMARIZATION_SYSTEM_PROMPT :
```

You are summarizing a conversation for context management. Preserve all critical technical decisions, code snippets, and user requirements. Be concise but complete. The summary will be sent to the LLM as-is.

It is sent via `completeSimple()` — non-streaming — because the summary's token count has to be known before continuing. The summarization call uses the same model as the session (unless overridden in settings), which means the summary cost is proportional to the session cost. A cheaper model could summarize, but the author opted for consistency: the summary is in the model's native style.

What the prompt avoids saying is "summarize the conversation concisely." LLMs hate that instruction because it invites impressionistic summaries that drop precisely the technical content you need. Naming the things to preserve — decisions, code, requirements — skews the model toward structured fidelity.

What gets kept intact

The compaction entry stores `firstKeptEntryId`. Everything from `firstKeptEntryId` up to (but not including) the compaction is *preserved verbatim* in the LLM context.

This matters because the most recent messages are what the model needs to continue correctly. A summary says "we decided X, then Y"; the actual tool-result messages say "I just ran `edit` and here is the diff." The diff matters for the next turn; the summary doesn't have that detail.

Exactly how many messages get preserved depends on settings but is typically the last few turns — enough context for the model to know "where it was" without dragging in all the old junk.

Inserting the compaction entry

The compaction is a regular JSONL entry (`src/core/session-manager.ts:869-889`):

```
appendCompaction({
  summary: "...",
  firstKeptEntryId: "m17",
  tokensBefore: 180_432,
  details: ...,
  fromHook: false,
})
```

Parent is the current leaf. After append, the compaction is the new leaf. New messages will have the compaction as their parent.

Because the compaction is an entry, not a rewrite, **no history is lost**. The old messages are still in the file. `/tree` can still navigate into pre-compaction state. You could, in principle, fork back to before the compaction and continue without summarization (you just couldn't actually fit it into the model's context).

Context reconstruction with compactions in it

When the next LLM call needs the context, the tree walker (`src/core/session-manager.ts:385-408`) emits:

1. The compaction summary as a synthetic user message at the position of the compaction. (Why a user message? Because system prompt is already set; a user message is the cleanest way to inject a chunk of context the LLM will treat as established history.)
2. All entries from `firstKeptEntryId` up through the compaction's parent — the preserved recent history.
3. All entries *after* the compaction — whatever new stuff has happened since.

The LLM sees: "[summary of earlier conversation] [last N turns verbatim] [new user message]". This is the minimum viable context for continuity.

Compaction and branches

What happens if you `/fork` from a message, one branch compacts, and the other doesn't? Nothing weird. The compaction entry is in one branch's path to leaf but not the other's. Walking the other branch's tree gives the full, uncompacted history.

This is not a special-case; it's a consequence of compaction being an entry. Entries are in whatever branches contain them. Branches are paths to leaves. Simple.

The overflow-recovery loop

The specific flow for "context overflowed mid-turn":

```

agent.prompt("hi")
agentLoop → stream
stream emits { type: "error", reason: "error", error: message }
    message.errorMessage matches an OVERFLOW_PATTERN
isContextOverflow(message, model) → true
session detects overflow
_overflowRecoveryAttempted ? abort : true
compact()
agentLoopContinue (last message is not assistant, so this is valid)
stream retries with smaller context

```

`_overflowRecoveryAttempted` is the infinite-loop guard. If we compact, retry, and still overflow, we emit the error and stop. The user can `/compact` more aggressively manually, or switch to a larger-context model.

Failures to watch for

- **Summary blows through reserve.** A model occasionally writes a summary too long to leave real room for new tokens. Settings allow tuning `branchSummary.reserveTokens` separately. Watch your logs.
- **Summarization itself overflows.** If the session is already so big that the *summarization call* can't fit, compaction fails. The session is stuck. The fallback is to manually `/fork` from an earlier point and start a new branch.
- **Important context not in the summary.** Heuristic: if the model repeatedly asks about details that were established pre-compaction, the summary isn't doing its job. The solution is usually to shrink `reserveTokens` so more recent messages stay verbatim, or to keep critical files in a `MEMORY.md` that gets appended to the system prompt.

What compaction does not touch

- Tool call IDs and their results: compaction never decouples a tool call from its result in a way that would make the conversation invalid. The summary replaces both.
- Thinking blocks in preserved messages: kept as-is. Nothing strips them.
- Session files on disk: never rewritten. Compaction is always an append.

In one paragraph

Compaction is: detect "we're close to the context limit," call a small LLM round-trip to summarize old history, insert a `compaction` entry into the session tree that references where "new" context starts, and rely on the tree walker to emit summary + preserved-recent

- new when building the next LLM context. Triggers are auto-by-size, manual, and overflow-recovery (capped at one retry). The summary prompt names the things to preserve explicitly. No history is ever deleted; compaction is just another kind of entry.

Next: extensions, where "anything the core doesn't do" lives.

Extensions: trust, lifecycle, API

The founding philosophical move of pi-coding-agent is that MCP, plan mode, subagents, and memory files are not features — they are extensions. Anything beyond the four built-in tools and the JSONL session has to earn its way into the core; until then, it lives as TypeScript a user has installed.

This isn't an "also" feature. A full day of using `pi` might involve half a dozen active extensions, any of which could override tool behavior, inject new tools, register slash commands, handle keyboard shortcuts, or replace the system prompt.

What an extension is

A TypeScript file (or a directory with an `index.ts` / `index.js` entry). Example:

```
// ~/.pi/extensions/pin-files.ts
import type { Extension } from "@mariozechner/pi-coding-agent";

export default <Extension>{
  name: "pin-files",

  async init(ctx) {
    const pinned = new Set<string>();

    ctx.registerSlashCommand("pin", "Pin a file", async ({ args }) => {
      pinned.add(args);
      return `Pinned ${args}`;
    });

    ctx.on("turn_start", (ev, { session }) => {
      if (pinned.size === 0) return;
      session.appendNextTurnMessage({
        role: "user",
        content: [{ type: "text",
          text: `Pinned files:\n${[...pinned].map(p => "- " + p).join("\n")}` }],
      });
    });
  },
};
```

It's a JavaScript object with a `name` and an `init` function. Inside `init`, the extension uses a context object (`ctx`) to register things and subscribe to events.

Discovery

Two auto-scanned locations:

location	purpose
<code>~/.pi/extensions/</code>	User-global. Loaded in every project.
<code>./.pi/extensions/</code>	Project-scoped. Loaded only in this project.

Plus CLI overrides:

flag	effect
<code>--extension=path</code>	Load a specific file. Repeatable.
<code>--no-extensions</code>	Disable auto-discovery entirely.

Resource loading lives in `src/core/resource-loader.ts` and extension loading in `src/core/extensions/loader.ts`.

How TypeScript extensions run without compiling

`pi` bundles `@mariozechner/jiti`, a TypeScript runtime loader that transpiles TS on import. The loader at `src/core/extensions/loader.ts:58-87` creates a `jiti` instance with import aliases:

```
const jiti = createJiti({
  alias: {
    "@mariozechner/pi-coding-agent": /* bundled path */,
    "@mariozechner/pi-ai": /* bundled path */,
    "@sinclair/typebox": /* bundled path */,
    // ...
  },
});
```

Why aliases? So an extension that says `import { Agent } from "@mariozechner/pi-coding-agent"` gets the already-loaded version in the running process, not a second copy it would have to resolve via `node_modules`. Two copies would fail `instanceof` checks; one copy means the extension shares types, classes, and state with the host.

In Bun binary mode (when `pi` has been compiled to a single executable via `bun build --compile`), file resolution doesn't work. The loader (lines 43-50) switches to `virtualModules` — a map of in-memory module definitions. Extensions that depend on a module outside this map won't work in binary mode. This is an intentional tradeoff: the binary is easier to ship, and extensions that need arbitrary `node_modules` can just be used from source.

The trust model

There is no sandbox. The loader is `jiti.import()`. The extension runs in the same Node.js process, with the same permissions, as `pi` itself. An extension has:

- File system access via `fs`

- Network access via `fetch`, `http`, etc.
- Shell execution via `child_process`
- Full pi-coding-agent API access — session state, mutating messages, changing the model, reading auth storage
- No confinement whatsoever

This is chapter 3's rule 7. It is deliberate and it is the right choice for pi's audience and threat model. Other agents make the opposite choice (Cursor runs extensions in a V8 isolate with a capability system). pi does not.

The practical implication: **install extensions the way you install npm packages — by looking at what they do first**. Don't install random extensions. The `CONTRIBUTING.md` and README both say this implicitly by treating extensions as "your code."

The API surface

What an extension can register or subscribe to (from `src/core/extensions/` and `src/core/extensions/loader.ts:161-200`):

Lifecycle events

- `session_before_switch` / `session_before_fork` — before a session handover. Chance to persist.
- `turn_start` — before each LLM call.
- `turn_end` — after each assistant message completes, including tool results.
- `message_start` / `message_update` / `message_end` — streaming message lifecycle.
- `tool_execute_start` / `tool_execute_update` / `tool_execute_end` — tool lifecycle.
- `compaction_start` / `compaction_end` — before and after a compaction fires.
- `agent_start` / `agent_end` — outermost. Only meaningful for long-running extensions.

Registrations

- `registerTool(tool)` — define a new `AgentTool` the LLM can call.
- `registerCommand(name, opts)` — register a CLI-style command the user can invoke by typing `/<name> ...`.
- `registerSlashCommand(name, desc, fn)` — simpler sugar for slash commands that just take one string arg.
- `registerShortcut(keyId, opts)` — bind a keyboard shortcut in interactive mode, with a label for the help menu.
- `registerProvider(name, cfg)` — register a custom LLM provider entry. Useful if you have a provider pi-ai doesn't ship.
- `registerMessageRenderer(type, fn)` — custom renderer for custom message types.
- `registerToolRenderer(toolName, fn)` — custom renderer for a specific tool's output in the TUI (e.g., a pretty-printer for your custom `spreadsheet` tool).

Session manipulation

Via the `session` binding on the `ctx`:

- `session.addTool(tool)` / `session.removeTool(name)` — dynamically add/remove tools mid-session (e.g., enable MCP tools only when an MCP server is online).
- `session.appendNextTurnMessage(msg)` — inject a message that will be sent as user context on the next turn.
- `session.appendCustomMessage(entry)` — write a custom entry to the session JSONL.
- `session.compact()` — force a compaction.
- `session.abort()` — cancel the running turn.

UI primitives

Via `ctx.ui`:

- `ctx.ui.select(options)` — open a select overlay, await a choice.
- `ctx.ui.confirm(message)` — yes/no prompt.
- `ctx.ui.input(prompt)` — single-line input.
- `ctx.ui.showError(...)`, `ctx.ui.showInfo(...)` — toast messages.

These work in all modes. In interactive mode they use pi-tui overlays. In RPC mode they emit `extension_ui_request` events to the client, which handles them. In print mode they usually throw or auto-resolve to a default (because there's no user present).

Lifecycle

Startup

- └ loader discovers files in ~/.pi/extensions/ and ./pi/extensions/
- └ for each file:
 - └ jiti.import(file) → extension object
 - └ stub ctx methods that throw "not initialized"
 - └ extension.init(ctx) runs (registrations stored but not active)
- └ bindCore(extensions, runtime)
- └ replace stubs with live implementations
- └ merge registered tools into the session
- └ merge registered commands/shortcuts into the TUI
 - └ subscribe event handlers
- └ extensions are now live

Session switch / fork

- └ emit session_before_switch
- └ call extension.destroy(oldCtx) if defined
- └ call extension.init(newCtx) for new session
- └ bindCore again

Shutdown

- └ emit agent_end
- └ call extension.destroy() for each extension
- └ process exits

The stub-then-replace pattern is notable (`src/core/extensions/loader.ts:134-151`). During `init()` , the extension can register things, but it cannot yet *call* session actions — because there's no session bound yet. Calling `ctx.session.compact()` during `init()` will throw "not initialized." This catches a class of bugs where an extension tries to do work before the core is ready.

A real extension: MCP

The MCP integration is itself an extension (published separately). Sketch of what it does:

1. In `init()` , read a `.pi/mcp.json` config listing MCP servers.
2. Spawn each server as a child process, connect via stdio, handshake.
3. For each MCP server, list its tools, and call `session.addTool()` for each with an `execute` that delegates to the MCP call.
4. On `session_before_switch` , close connections. Reopen in the new session's init.

No core changes needed. The built-in tools are unchanged. The session JSONL format is unchanged. The agent loop is unchanged. MCP is just a set of tools that happen to proxy to another process.

A real extension: Plan mode

"Plan mode" in other agents is often a first-class feature. In pi, it's an extension:

1. In `init()` , register a `/plan` slash command and a keyboard shortcut.
2. When the user invokes it, switch to a specific system prompt that asks the LLM to produce a plan, not edit files.

3. Lock out the `edit`, `write`, and `bash` tools for the duration (via `session.removeTool` and restoring on `turn_end`).
4. On each `turn_end` while in plan mode, check for a "ready to execute" signal and switch back.

Again, no core changes.

Isolation by convention, not by force

Because extensions are not sandboxed, it is *possible* for a misbehaving extension to break other extensions or the core. Pi's conventions are:

- **Don't mutate shared state** (`session.messages`, etc.) directly. Use the provided session methods.
- **Don't monkey-patch anything**. If you need to alter behavior, register a hook.
- **Namespace your events** if you emit them. `pin-files.event-name`, not just `event-name`.

These are not enforced. But extensions in the wild all follow them because violating them has an obvious cost: your extension is the one that breaks next time someone upgrades.

Practical notes

- **Hot reload is not supported**. Changing an extension file requires restarting `pi`.
- **Error handling**. If an extension throws during `init()`, `pi` logs the error and continues without it rather than refusing to start. Better to have a partial `pi` than none.
- **Performance**. Extension event handlers run inline in the agent loop. A slow handler applies backpressure (via the Agent class's await-subscribers model). Keep them fast or make them async and fire-and-forget.

In one paragraph

Extensions are TypeScript files loaded at startup from `~/.pi/extensions/` and `./.pi/extensions/`, transpiled by jiti, running in the same process with no sandbox. They register tools, commands, shortcuts, providers, renderers, and handlers via a small API, and they have full access to the session and all of pi. The trust model is "don't install untrusted extensions"; the payoff is that anything the core doesn't do (MCP, plan mode, memory files, custom providers, remote shells) can be added in a file under 200 lines.

Next: how pi stores the credentials you give it, and how OAuth fits in.

Auth, credentials, and OAuth

Pi has to hold credentials for twenty-plus providers. Some are static API keys; some are OAuth tokens with refresh flows; some (Bedrock) are whole AWS credential chains. This chapter covers the storage format, the refresh semantics, and the runtime override escape hatch.

The storage file

Everything lives in `~/.pi/auth.json`. A flat JSON object keyed by provider id:

```
{
  "anthropic": { "type": "api_key", "key": "sk-ant-..." },
  "openai":     { "type": "api_key", "key": "sk-..." },
  "google":     { "type": "oauth", "accessToken": "...", "refreshToken": "...", "expires":
1745208000000 },
  "copilot":    { "type": "oauth", "accessToken": "...", "refreshToken": "...", "expires":
... },
  "bedrock":    { "type": "aws", "profile": "default", "region": "us-east-1" }
}
```

File permissions are explicitly `chmodded` to `0600` (`src/core/auth-storage.ts:57, 100`). Owner-only read and write. That's the entire at-rest security model: **don't let other users on the machine read your home directory**. There is no encryption at rest; the author considered a passphrase-protected store and decided it wasn't worth the UX cost.

The `AuthStorage` abstraction

`src/core/auth-storage.ts` exposes two implementations:

- `FileAuthStorageBackend` — the normal one. Reads, writes, locks the file with `proper-lockfile`.
- `InMemoryAuthStorageBackend` — ephemeral. Used in tests and in `--no-session` mode where everything is supposed to disappear when the process ends.

Both implement `AuthStorageBackend` with `withLock()` and `withLockAsync()` helpers so call sites don't have to remember to release locks.

OAuth: the supported providers

Five OAuth flows in `pi-ai` (`packages/ai/src/utils/oauth/index.ts:35-48`):

provider	flow	token use
Anthropic	Claude Pro / Max OAuth	Converts to an API key equivalent for the session.
GitHub Copilot	Device flow	Bearer token against the Copilot chat API.
Google Gemini CLI	gcloud auth refresh	Uses the same tokens <code>gcloud</code> uses.
Google Antigravity	Google Cloud Code Assist OAuth	Enterprise flow.
OpenAI Codex	ChatGPT OAuth	Treats your ChatGPT login as an API key.

For each, pi implements (a) an initial flow (browser → callback, usually), (b) a refresh flow that exchanges a refresh token for a new access token, and (c) a translation step that turns the access token into whatever the provider actually wants on each HTTP call.

The refresh loop

`getOAuthApiKey()` at `packages/ai/src/utils/oauth/index.ts:119-162`:

```
export async function getOAuthApiKey(
  providerId: OAuthProviderId,
  credentials: Record<string, OAuthCredentials>,
): Promise<{ newCredentials: OAuthCredentials; apiKey: string } | null> {
  let creds = credentials[providerId];
  if (!creds) return null;

  if (Date.now() >= creds.expires) {
    creds = await provider.refreshToken(creds);
  }

  return { newCredentials: creds, apiKey: provider.getApiKey(creds) };
}
```

The contract is: the caller passes in the current credentials, and the function returns potentially-new credentials and the usable `apiKey` for now. **If new credentials came back, the caller must persist them.** This is not enforced — it's a property of the API that the coding agent has to honor.

The coding agent honors it through `AuthStorage.refreshIfNeeded()`, which calls `getOAuthApiKey()` and immediately writes the new credentials back to `auth.json` under lock.

Runtime overrides

`main.ts:566-575`: the `--api-key` CLI flag sets a session-only override. It takes precedence over whatever is in `auth.json`. It does not get persisted. Usage:

```
pi --api-key sk-... "fix the readme"
```

This is useful for CI pipelines where the API key is in an env var, or for quickly trying a different key without contaminating the stored one.

There's also an env-var scanning layer in pi-ai (`packages/ai/src/env-api-keys.ts`) that looks for provider-specific env vars (`ANTHROPIC_API_KEY` , `OPENAI_API_KEY` , `GOOGLE_API_KEY` , etc.) and treats them as implicit overrides. Priority order:

1. Explicit `--api-key` flag.
2. Provider-specific env var.
3. Stored credentials in `auth.json` .

Nothing found → pi prompts for the key interactively (when TTY), or errors out (when not).

The login subcommand

`pi login [provider]` opens whichever flow the provider supports. For OAuth providers this launches a browser window to the authorization URL and starts a local callback server. For API-key providers it prompts for the key on the command line.

The stored credentials are what future `pi` invocations use. `pi logout [provider]` removes them.

Per-provider env auto-detection

`pi-ai` also includes a credential auto-detection layer for Bedrock that inspects `AWS_PROFILE` , `AWS_REGION` , `AWS_ACCESS_KEY_ID` , etc. in that order, building up a credential chain via the AWS SDK's default chain. You don't explicitly "log in" to Bedrock with pi; you set up your AWS creds normally and pi finds them.

What pi does not do

- **No keyring integration.** On macOS you can store credentials in the system Keychain, on Linux in the Secret Service, on Windows in Credential Manager. Pi doesn't. It's plaintext on disk. The tradeoff is "works everywhere the same way," and the counter-mitigation is file permissions.
- **No per-project scoping.** `auth.json` is always global. You can't have "this project uses this Anthropic key, the other project uses that one." The workaround is env vars, which scope naturally to whatever process you launch.
- **No expiring tokens for API keys.** API keys are assumed static. Provider-side rotation (rotate an Anthropic key every 90 days) requires you to `pi login anthropic` again.

In one paragraph

Credentials live in `~/.pi/auth.json` , `chmod 600`, locked with `proper-lockfile` for concurrent `pi` instances, and split by provider with a discriminated-union type covering API keys, OAuth tokens, and AWS profiles. OAuth refreshes are driven by a caller-persist contract: the refresh function returns new creds, and the

caller writes them back. Runtime overrides via `--api-key` and env vars sit in front of the stored values with a defined priority.

Next chapter pulls the modes apart: interactive, print, and RPC.

Three ways to run the agent

The coding agent ships three modes over the same `AgentSession` core: interactive, print, and RPC. There's also a fourth, `json`, which is really a variant of print. The modes differ almost entirely in I/O — how prompts come in and how events go out. Everything below the I/O is identical.

Interactive (TUI)

The mode you get when you run `pi` in a terminal with no flags. Source: `src/modes/interactive/`.

Flow:

1. Initialize pi-tui.
2. Render the conversation view with a message list and an editor at the bottom.
3. Subscribe to `AgentSession` events; update the view on each event.
4. On Enter in the editor, call `session.prompt(text)`.
5. On an arriving `tool_execution_start` where `beforeToolCall` requires approval, pause the session and overlay a confirmation dialog.
6. Handle keybindings via `KeybindingsManager`.

Key UI pieces:

- **Editor** (pi-tui's multi-line component). Typed history, Emacs- style kill ring, undo, autocomplete for file paths and slash commands.
- **Message list** renders per message type. Text renders as Markdown; tool calls render as labeled panels with the tool's pretty output; thinking blocks render as collapsible sections.
- **Status line** at the top: current model, thinking level, token usage, cost.
- **Overlays**: model selector (Ctrl+P), session tree (double Esc), settings, extension UI dialogs.

Interactive mode is the largest part of the codebase outside of tools and providers, and most of its code is pi-tui components and layouts. The TUI does a lot of work here; interactive mode is mostly a glue layer between `AgentSession` events and pi-tui updates.

Print mode

Single-shot. `pi "fix the readme typo"` runs one conversation and exits. Source: `src/modes/print-mode.ts`.

Flow:

1. Read the prompt from argv, `@file.txt`, or stdin if piped.

2. `session.prompt(text)`.
3. Subscribe to events, accumulate the final assistant text.
4. On `agent_end`, write the final text to stdout and exit.

Print mode has a subtle twist: if any tool wants approval, print mode has no user to ask. By convention, the default extension config for print mode auto-approves read-only tools (`read`, `grep`, `find`, `ls`) and rejects everything else. You can override per invocation with flags.

`--mode json` is print mode with a different output: instead of accumulating text, each event is written as a JSON line to stdout as it arrives. You get the full event stream. This is what CI pipelines and scripts hook into.

RPC mode

Stdin/stdout JSON-RPC. Source: `src/modes/rpc/`. Used by:

- IDE extensions (VS Code, potentially others).
- pi-mom, when running pi as a subprocess instead of linking the SDK.
- Any external orchestrator.

Protocol (`src/modes/rpc/rpc-types.ts:19-68`):

Commands come on stdin, one per line:

```
{ "id": "req-1", "type": "prompt", "message": "Hello", "images": [] }
{ "id": "req-2", "type": "abort" }
{ "id": "req-3", "type": "set_model", "provider": "anthropic", "model": "claude-sonnet-4-6" }
{ "id": "req-4", "type": "get_state" }
{ "id": "req-5", "type": "fork", "entryId": "msg-42" }
{ "id": "req-6", "type": "switch_session", "path": "..." }
{ "id": "req-7", "type": "bash", "command": "ls" }
```

Every command has an `id` for correlation. Full list includes: `prompt`, `steer`, `follow_up`, `abort`, `set_model`, `cycle_model`, `get_available_models`, `get_state`, `get_messages`, `get_session_stats`, `bash`, `switch_session`, `fork`, `new_session`.

Responses echo the id and a success/error:

```
{ "id": "req-1", "type": "response", "command": "prompt", "success": true }
```

Events are the agent's event stream, emitted as they happen, with no `id` (they aren't responses to anything):

```
{ "type": "message_start", ... }
{ "type": "tool_execute_start", "name": "bash", ... }
{ "type": "message_end", ... }
```

Extension UI requests are emitted when an extension needs user input. The client should reply with a matching `extension_ui_response` :

```
// out
{ "type": "extension_ui_request", "requestId": "ui-1", "kind": "confirm", "message":
  "Allow bash?" }
// in
{ "type": "extension_ui_response", "requestId": "ui-1", "result": { "confirmed": true } }
```

Default values and timeouts are supported so a client that disappears mid-dialog doesn't hang the session forever.

The `takeOverStdout` guard

From chapter 7: `main.ts:451` redirects `console.log` and `console.error` to internal buffers when running in print, json, or RPC modes. Otherwise a stray log from an extension would corrupt the output protocol. The buffers are flushed to stderr on exit so you can still see them for debugging.

Mode-specific extension UI

The same `ctx.ui.confirm()` call works in all three modes because the UI primitives are abstract. Behind the scenes:

mode	how confirm() works
interactive	Opens a pi-tui overlay with Y/N buttons; awaits keypress.
print	Uses the configured default (often auto-approve); no user input.
json / rpc	Emits <code>extension_ui_request</code> ; awaits matching response.

Extensions write against the abstract API and work everywhere.

Why three modes

Why not just pick one?

- **Interactive** is what humans want when they sit at a terminal and want to see a rich rendering.
- **Print** is what scripts and CI want. Machine-parseable output, no TTY requirements, stdin-pipe-able.
- **RPC** is what IDE integrations and orchestrators want. Full event access, the ability to issue arbitrary commands, and a symmetric command/event protocol so client and server can evolve independently.

None of the three is a strict subset of another. Removing any one would remove a user.

Switching modes mid-session

You can't. Modes are chosen at process start and the I/O wiring is different enough that switching would require re-architecting the event emission path. If you want to go from interactive to RPC, you exit and restart.

However, a session file can be consumed by any mode. `pi --resume my-session` will pick up in whatever mode you're launching now, regardless of what mode created the session. Session format is mode-agnostic.

In one paragraph

Three modes — interactive (TUI), print (one-shot stdout), RPC (stdin/stdout JSON) — sit over the same AgentSession core. They differ only in I/O: what reads the prompt, how events render, how UI dialogs are resolved. The abstraction is a handful of UI primitives that each mode implements concretely, so extensions written against the abstract API work everywhere. Session files don't care which mode wrote them.

Part IV starts with the Slack bot.

PART IV

The ecosystem

pi-mom — the agent on Slack

"mom" is a Slack bot that wraps pi-coding-agent and lets you talk to it from a channel. The name is a joke: it's like the mom you can ask to fix things while you're at work. The implementation is surprisingly small — under 2,500 lines — and it's a good study of "how do you wrap the coding agent in a different UI without forking it."

The three actors

`packages/mom/src/main.ts` boots three things:

1. **SlackBot** (`slack.ts:153-169`), which opens a Socket Mode connection to Slack. Socket Mode means pi-mom holds a persistent WebSocket to Slack rather than exposing an HTTP webhook — works behind NAT, no public URL required.
2. **EventsWatcher** (`events.ts`), a file-system watcher for scheduled/timer-driven tasks.
3. **Per-channel AgentRunner** instances (`agent.ts:392-410`), cached in memory. Each channel gets one runner. The runner owns a long-lived `AgentSession` bound to that channel's workspace.

The main handler (`main.ts:281-332`) is a non-blocking dispatcher. Slack events come in, get logged synchronously, then either get rejected (if the channel's runner is busy) or pushed to the channel's message queue.

How pi-mom talks to pi

It doesn't shell out, doesn't use RPC. It **calls the coding-agent SDK in-process** (`agent.ts:467-476, 780`):

```
import { createAgentSession } from "@mariozechner/pi-coding-agent";

const session = await createAgentSession({
  cwd: channelWorkspaceDir,
  authStorage,
  extensions: [...],
});

await session.prompt(userMessage, { images });
```

It's the same SDK any embedding would use. The advantage over RPC is no serialization overhead, no subprocess management, and direct access to the event stream. The disadvantage is that mom's memory and crash behavior are tied to pi's — if pi crashes, mom crashes too.

Per-channel sessions

Each Slack channel gets its own workspace directory (`/data/<channel-id>/`), its own session file (`context.jsonl`), its own memory file, and its own skills directory. The `AgentRunner` (`agent.ts`) wraps

this setup.

The per-channel isolation is meaningful: two channels discussing unrelated projects get unrelated agent state. Switching channels is switching sessions. Files written in one channel stay in that channel's workspace and aren't visible to another.

Summary in channel, details in thread

The split is the bot's most distinctive UI choice. For every response from the agent:

- A **summary** goes to the main channel — short, what-happened-at-a-glance. Truncated to 35 KB if needed.
- The **details** (tool calls, full output, intermediate steps) go to the thread attached to the message.

Implementation in `agent.ts:499-620`. The handler subscribes to `message_end` events, partitions the content into main-channel text and thread-detail text, and enqueues two posts.

Tool executions in the thread look like:

```
🔧 bash
$ find . -name "*.py" | head
./foo.py
./bar.py
```

While the main channel just shows: `→ bash` and then the next text.

The message queue (`agent.ts:702-728`) is a promise chain that serializes all Slack API calls. Slack's rate limits are real; a naive concurrent poster would get throttled. The chain guarantees in-order delivery and respects API backoff.

Editing the final message

When the agent is still running and streaming, pi-mom posts a placeholder ("thinking..."). When the turn ends, it **edits** the placeholder via `chat.update` to contain the final summary. Source: `agent.ts:819`.

This is much nicer than "here is my initial thought... actually, here is my updated thought... actually, my final thought is..." which is what you get if you post each token as it arrives. The message list stays tidy.

Docker sandbox

Mom accepts `--sandbox=host` (run tools on the host) or `--sandbox=docker:<container>` (run them in a container).

The `DockerExecutor` (`sandbox.ts:179-193`) wraps every bash call:

```
docker exec <container> sh -c "<command>"
```

With path translation: the host workspace `/data/CHANNEL-ID/` corresponds to `/workspace/CHANNEL-ID/` inside the container. Tool calls that read or write files against the host path get rewritten.

Stdout and stderr are capped at 10 MB per stream (`sandbox.ts:141, 147`) so a misbehaving `cat /dev/zero` can't eat memory.

No host directories are automatically mounted; the operator provisions the container (and mounts whatever they want) externally via a setup script. Mom just runs `docker exec` against it.

This is not a security boundary against a motivated adversary — the container has shell access to whatever its mounts and image allow. It is a **blast-radius** limiter. A runaway bash command can wreck the container; it can't wreck the host.

Skills and memory

Mom loads two kinds of persistent context on every run:

- **Skills** (`agent.ts:105-139`): `workspace/skills/<name>/SKILL.md` files with Markdown frontmatter. Workspace-level skills apply globally; per-channel skills override. Loaded and listed in the system prompt on each run.
- **Memory** (`agent.ts:69-103`): `MEMORY.md` file(s) that get appended to the system prompt verbatim. Workspace-level applies globally; per-channel overrides.

The agent has `read`, `write`, and `bash` access to its workspace. It can create and modify skills and memory files by itself. "Remember that we always use black formatting" → write it to `MEMORY.md`. "Here's a new playbook for running migrations" → create `skills/migrations/SKILL.md`.

This self-configuration is the piece that makes pi-mom feel live-trainable. You teach it something by chatting; it persists the lesson; next session it applies automatically.

Deduplication and backfill

Slack Socket Mode connections drop and reconnect. When that happens, Slack re-delivers events that arrived during the outage. The `ChannelStore` (`store.ts:33-50`) tracks logged timestamps in a 60-second window to reject duplicates.

Separately, on startup, mom backfills `log.jsonl` — the persistent event log on disk — against the current session's known state. If a user message came in while mom was down and got logged to `log.jsonl`, the session on startup catches up before taking new input.

Credentials

Two env vars required (`main.ts:16-17`):

- `MOM_SLACK_APP_TOKEN` — Socket Mode app-level token.
- `MOM_SLACK_BOT_TOKEN` — the bot user's workspace token.

Both are read at startup and never logged. LLM API keys live separately in `~/.pi/mom/auth.json` — **not** the default `~/.pi/auth.json` — so a compromised mom workspace can't read the operator's personal pi credentials. Nice touch.

Abort semantics

If a user says something that mom interprets as "stop" (or the user invokes a stop command), the current run calls `session.abort()`, which sets the AbortSignal on the whole event stream. Tools get the signal, HTTP requests tear down, the agent exits with a `turn_end` + `agent_end`, and the "thinking" message in the channel gets edited to "aborted." No forked state, no dangling tool calls.

In one paragraph

Pi-mom is a Slack bot that embeds pi-coding-agent via its SDK, gives each Slack channel its own long-lived session and workspace, optionally runs tool calls inside a Docker container for blast-radius containment, and teaches itself via SKILL.md and MEMORY.md files the agent can read and write. The clever UI move is summary-in-channel / details-in-thread, implemented as a promise-chain message queue over the Slack Web API. Its existence is a proof that the coding-agent SDK is genuinely embeddable.

Next: the web-ui package.

pi-web-ui — agents in the browser

`packages/web-ui` is the one package in pi-mono that doesn't run in a terminal. It's a library of [Lit](#) web components for building chat-style agent UIs in a browser, plus a handful of artifact renderers for PDFs, spreadsheets, Word documents, Markdown, and sandboxed HTML.

It is unusually broad for a 5 k-LOC package because "a full agent UI" has a lot of moving parts. This chapter tours what's there and how it fits together.

The component library

The default entry point is `ChatPanel` — a `LitElement` that composes `AgentInterface` (the chat) and `ArtifactsPanel` (the output renderer) into a two-pane layout.

From there, granular components are exported for apps that want their own layout:

- `AgentInterface` — the chat scrollbar, streaming message container, input editor.
- `MessageList` / `StreamingMessageContainer` — render an array of messages and a currently-streaming one.
- `MessageEditor` — the input component, with file attachment support.
- `ModelSelector`, `SettingsDialog`, `SessionListDialog`, `CustomProviderDialog`, `ApiKeyPromptDialog` — the modals.

And the artifact renderers:

- `HtmlArtifact` (via `SandboxedIframe`)
- `PdfArtifact` (via `pdfjs-dist`)
- `MarkdownArtifact`
- `ExcelArtifact` (via `xlsx`)
- `DocxArtifact` (via `docx-preview`)
- `SvgArtifact`, `ImageArtifact`

Peer dependency on `lit@^3.3.1`. An alternative lightweight path exists via `@mariozechner/mini-lit`. The library ships unbundled ES modules, so apps choose their bundler.

Talking to the LLM from the browser

`AgentInterface` accepts an optional `session?: Agent`. Two patterns:

1. **Bring your own `Agent`**. The host app creates an `Agent` (from pi-agent-core) and passes it in. Events flow from the agent into the component; user input from the component into the agent. The example app at

`packages/web-ui/example/` does this.

2. **Provider-direct.** Without a session, the component uses pi-ai's `streamSimple` directly — in-browser `fetch` to provider APIs, with a proxy-bypass layer for CORS (`utils/proxy-utils.ts`). Credentials come from an in-browser IndexedDB-backed `ProviderKeysStore`.

The second pattern is how a static-hosted web app can be a full agent UI with no backend. You paste an Anthropic API key, it goes into IndexedDB, and from then on `fetch` calls go directly from the user's browser to `api.anthropic.com`. Your host never sees the messages.

Two things to note:

- The API key is in the browser. A malicious extension or compromised third-party script can steal it. Not for production apps handling sensitive data.
- Provider APIs don't always ship permissive CORS headers. The `proxy-utils` layer routes through a small proxy when needed — typically a Cloudflare Worker or similar that the deployer runs.

The artifact system

An artifact is a piece of content the agent produces that isn't chat — a generated HTML page, a calculated Excel sheet, a rendered Markdown doc. The `ArtifactsPanel` shows a list of artifacts; each has its own renderer.

The twist: **HTML artifacts run in a sandboxed iframe with a postMessage bridge** (`SandboxedIframe.ts`, 600+ lines, the largest file in the package). The iframe has `sandbox="allow-scripts"` but not `allow-same-origin`, so its JS can run but it can't touch the parent's DOM, cookies, or storage.

A message bridge exposes *runtime providers* to the iframe:

- `ArtifactsRuntimeProvider` — create/read/update other artifacts from inside the iframe.
- `AttachmentsRuntimeProvider` — access user-attached files.
- `ConsoleRuntimeProvider` — logging that surfaces in the host.
- `FileDownloadRuntimeProvider` — save a file to the user's disk.

This means the agent can generate rich interactive HTML — a dashboard, a calculator, a visualization — and it runs in your browser without seeing your credentials, your chat history, or your other artifacts unless you explicitly expose them.

Document rendering

The file-type artifacts each wrap a third-party library:

format	library
PDF	<code>pdfjs-dist@5.4.394</code> — renders pages to canvas, paginates.
XLSX	<code>xlsx</code> (SheetJS) — parses and renders as an HTML table.
DOCX	<code>docx-preview@0.3.7</code> — renders into an HTML container.
Markdown	Native Lit template.
HTML/SVG	Sandboxed iframe.
Image	<code></code> with base64 data URL.

The tool flow: the agent generates content, calls a tool like `create_artifact` (provided by the example app), and the resulting `custom_message` in the session causes the web UI to render it in the artifacts panel.

Persistence

Everything persists in IndexedDB:

- `SessionsStore` — the conversation history (mirrors the coding agent's JSONL on disk but in browser).
- `SettingsStore` — UI settings, theme, default model.
- `CustomProvidersStore` — user-added custom OpenAI-compatible providers (e.g., pointing at a local Ollama).
- `ProviderKeysStore` — API keys.

Sessions can be exported as a `.jsonl` file, imported into pi the CLI, and vice versa. Same format.

Build and output

The library itself uses plain `tsc` — no bundler. Output is `dist/index.js`, `dist/index.d.ts`, and `dist/app.css` (Tailwind v4). Consumers bundle it themselves.

The example app in `packages/web-ui/example/` uses Vite for a dev server with HMR and a production build.

Tool renderers

Each tool result type can have a custom renderer. The built-in renderers:

- `BashRenderer` — syntax-highlighted terminal output.
- `DefaultRenderer` — fallback for any tool.
- `CalculateRenderer` — math output with MathJax.
- `GetCurrentTimeRenderer` — formatted time display.

Extensions register their own via `registerToolRenderer(name, fn)`. Same mechanism as the TUI in the coding agent; the underlying abstraction is shared.

What pi-web-ui does not do

- It doesn't ship a server. The library is all client-side; any server-side concerns (auth, multi-tenancy, shared sessions) are the host app's problem.
- It doesn't implement tools. The `AgentInterface` takes tools as input; it doesn't define them. This is deliberate: the web UI and the CLI should share tool definitions when possible, but the library stays agnostic about which tools exist.

In one paragraph

Web-ui is a Lit-based component library for building browser-side agent chat UIs, with rich artifact rendering (HTML sandboxed, PDF / XLSX / DOCX / Markdown / images native), IndexedDB persistence for sessions and credentials, and a peer-dependency model so apps choose their own Lit version and bundler. The headline feature is sandboxed HTML artifacts with a `postMessage` runtime-provider bridge — LLMs can generate rich interactive content that runs in the user's browser without seeing anything else.

Next: pi-pods.

pi-pods — rolling your own vLLM

pi-pods is the smallest of the satellite packages and the most "shell script wearing a suit." It exists because the author wanted to run open-weight models on rented GPUs and didn't want to remember the vLLM command-line flags every time.

What it is

A CLI that SSHes into GPU machines you already have, starts vLLM processes on them, and keeps a local registry of which models are running where. That's it. No Kubernetes, no cloud API integration, no orchestrator — it's `ssh` with a config file.

Supported targets: anything with SSH access. Runpod, Lambda Labs, Vast.ai, your own rack, a colo — pi-pods doesn't know or care. You give it an SSH command string and it uses that.

The architecture

```

~/pi/pods.json                (config, on your laptop)
|
|   pods.runpod-h200.ssh = "ssh -i key.pem user@ip"
|   pods.runpod-h200.gpus = [...]
|   pods.runpod-h200.models = { llama-70b: { port: 8001, pid, ... } }
|
pi start llama-70b             (on your laptop)
|
|   └─ reads pods.json
|   └─ picks a pod (default: "active" pod)
|   └─ picks GPUs (round-robin by current usage)
|   └─ builds vLLM args
|   └─ ssh <pod> "nohup python -m vllm.entrypoints.openai.api_server ..."
|       |
|       ▼
remote host spawns vLLM
updates pods.json with the new running-model entry

```

No daemon on your laptop. Each `pi-pods` invocation is stateless except for reads/writes to `pods.json`. No daemon on the remote either — vLLM just runs as a detached `nohup` process with its PID tracked.

Setting up a pod

```
pi pods setup my-pod "ssh -i key.pem user@1.2.3.4" --mount "/path/to/models"
```


The setup command (`commands/pods.ts`):

1. Validates SSH connectivity (`ssh <pod> echo ok`).
2. Runs `nvidia-smi --query-gpu=name,memory.total --format=csv,noheader` to detect GPUs.
3. Optionally runs a mount command if `--mount` is given.
4. Writes the pod entry to `~/.pi/pods.json`.

The GPU detection output becomes the pod's `gpus` array:

```
[
  { "id": 0, "name": "NVIDIA H200", "memory": "141GB" },
  { "id": 1, "name": "NVIDIA H200", "memory": "141GB" }
]
```

Starting a model

```
pi start meta-llama/Llama-3-70B-Instruct
```

`commands/models.ts` looks up the model in its database (`models.json` , loaded via `model-configs.ts`), which knows things like:

- Template name for vLLM.
- Required GPU count (a 70B parameter model needs 2× H200).
- Recommended flags (`--gpu-memory-utilization` , `--tensor-parallel-size` , `--max-model-len`).

It picks GPUs via round-robin: count current model usage per GPU, assign to the least-loaded GPU(s) that can fit. If nothing fits, it errors.

Then it SSHes this:

```
nohup python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3-70B-Instruct \
  --port 8001 \
  --tensor-parallel-size 2 \
  --gpu-memory-utilization 0.9 \
  > logs/model.log 2>&1 &
```

Captures the PID via `ps` , saves it to `pods.json` , prints the endpoint URL (`http://<pod-ip>:8001/v1`).

The model catalog

`model-configs.ts` contains a hard-coded list of models and their default configurations. Each entry looks roughly like:

```
{
  name: "llama-3-70b",
  huggingFaceId: "meta-llama/Llama-3-70B-Instruct",
  minGpus: 2,
  maxContextLen: 8192,
  extraArgs: ["--quantization", "awq"],
  family: "llama",
}
```

Add a new model = add a new entry. The list covers the common open-weight families (Llama, Qwen, Mistral, Mixtral, Yi, DeepSeek, GPT-OSS, and a scattering of fine-tunes).

vLLM version selection: pods supports three toggles — `"release"` (stable), `"nightly"` (main branch), `"gpt-oss"` (a specific fork needed for some models). Per-pod configuration.

Status and logs

```
pi list
pi logs my-pod
```

`list` enumerates running models across all pods (data straight from `pods.json`). `logs <pod>` opens an SSH tunnel and `tail -f` s the model's log file.

Stopping a model

```
pi stop my-pod
```

SSHes `kill <pid>` on the pod and removes the entry from `pods.json`. No grace period, no clean shutdown — vLLM handles SIGTERM fine in practice.

Hooking up to pi-ai

pi-pods does not auto-register its endpoints with pi-ai. After starting a model, you get an endpoint URL and manually add it as a custom provider:

```
// via an extension, or the web UI's custom provider dialog
registerProvider({
  name: "my-pod-llama-70b",
  baseUrl: "http://1.2.3.4:8001/v1",
  apiKey: "dummy", // vLLM requires a key but accepts any
  type: "openai-compatible",
});
```

From there, pi-ai treats it like any other OpenAI-compatible provider. The coding agent can use it; the web UI can use it; any pi-ai consumer can use it.

The `pi agent` shortcut

One convenience (`cli.ts:328-350`):

```
pi agent my-pod
```

Starts an interactive pi session pre-configured with the pod's running model as the provider. Handy for quickly chatting with a model you just spun up.

The `ssh.ts` helper

`src/ssh.ts` wraps `child_process.spawn("ssh", ...)` with:

- `sshExec(pod, command)` — capture stdout/stderr, return exit code.
- `sshExecStream(pod, command, { onData })` — stream output.
- `sshCopy(pod, localPath, remotePath)` — scp wrapper.

Nothing fancy. The whole file is under 100 lines.

What it's not

- Not a hosted service. You bring the machine.
- Not a model fine-tuner. It spins up vLLM for inference only.
- Not a secrets manager. The SSH key path is stored literally in `pods.json`; protect the file accordingly.

In one paragraph

Pods is a thin CLI that keeps a `pods.json` registry of GPU machines and the models running on them, SSHes into those machines to spawn `nohup`-detached vLLM processes, tracks PIDs and ports, and lets you list, log, and stop them. It doesn't integrate with cloud APIs or pi-ai automatically — you add endpoints as custom providers after starting them. The whole package is ~3 k LOC and reads like a polished shell script.

That's the seven packages. Part V covers adding to pi-mono: providers, tests, releases.

PART V

Going further

Adding a new LLM provider

If pi-mono lives or dies by "we talk to every LLM worth talking to," then adding a new provider needs to be a well-worn path. It is. The author documented it in AGENTS.md and the codebase has guardrails to make sure you don't forget a step.

This chapter walks the path. If you're adding a real provider while reading this, treat it as a checklist.

The seven-file tour

AGENTS.md's own list:

1. `packages/ai/src/types.ts` — add API id, options interface, KnownProvider name.
2. `packages/ai/src/providers/<name>.ts` — the implementation.
3. `packages/ai/package.json` + `src/index.ts` + `src/providers/register-builtins.ts` — exports and lazy registration.
4. `packages/ai/src/env-api-keys.ts` — credential detection.
5. `packages/ai/scripts/generate-models.ts` — model list generation.
6. `packages/ai/test/*.test.ts` — lots of them.
7. `packages/coding-agent/` — default model entry and env-var docs.

That's it. Seven places; none of them are architectural.

1. Core types

`packages/ai/src/types.ts` maintains three string unions:

- `Api` — every provider's API identifier. For example, `"anthropic-messages"`, `"openai-completions"`, `"openai-responses"`, `"bedrock-converse-stream"`.
- `KnownProvider` — the user-facing name.
- `ApiOptionsMap` — maps each `Api` to its options type.

Adding a provider is adding three entries (or subsets of them if a provider shares an API with another). Also add an options interface extending `StreamOptions`:

```

export interface FooStreamOptions extends StreamOptions {
  baseUrl?: string;
  customHeader?: string;
}

export type Api =
  | ...
  | "foo-chat";

export type ApiOptionsMap = {
  ...
  "foo-chat": FooStreamOptions;
};

export type KnownProvider =
  | ...
  | "foo";

```

2. Provider implementation

`packages/ai/src/providers/foo.ts` is the meat. It must export:

- `streamFoo(model, context, options)` — returns `AssistantMessageEventStream`.
- `streamSimpleFoo(...)` — a variant for `SimpleStreamOptions`.
- The `FooStreamOptions` interface (re-exported here for providers that want it).
- Helpers to convert `Context.messages` → provider native format.
- Helpers to convert `Context.tools` → provider native format.
- A response parser that consumes the provider's stream (SSE, WebSocket, or JSON-in-HTTP) and emits pi-ai events (`text`, `tool_call`, `thinking`, `usage`, `stop` / `error`).

The existing providers are templates. Start with whichever is closest in spirit:

- **OpenAI-compatible?** Copy `openai-completions.ts`. Most providers in the wild are OpenAI-compatible; this covers Groq, Cerebras, xAI, z.ai, MiniMax, AI Gateway, OpenRouter, and more.
- **Anthropic-style messages API?** `anthropic.ts`. Deep thinking, cache control, tool use blocks.
- **Google Gemini-style?** `google-gemini.ts`, or `google-gemini-cli.ts` if you need OAuth + reasoning interleaving.
- **Something novel?** `amazon-bedrock.ts` handles the Converse API; `openai-codex-responses.ts` handles Codex-flavored reasoning.

3. Registration and lazy loading

`packages/ai/package.json` needs a subpath export:

```
"exports": {
  ".": "./dist/index.js",
  "./foo": "./dist/providers/foo.js",
  ...
}
```

This lets downstream code do `import { streamFoo } from "@mariozechner/pi-ai/foo"`.

`packages/ai/src/index.ts` re-exports types consumers might want:

```
export type { FooStreamOptions } from "./providers/foo.js";
```

But **not the implementation functions**. Leave those lazy.

`packages/ai/src/providers/register-builtins.ts` is where the lazy registration happens. Add a module-level promise and wrapper factories:

```
let fooProviderModulePromise: Promise<FooProviderModule> | undefined;

const lazyStreamFoo = createLazyStream(
  () => (fooProviderModulePromise ||= import("./foo.js").then(m => ({
    stream: m.streamFoo,
    streamSimple: m.streamSimpleFoo,
  }))),
);

// in registerBuiltInApiProviders():
registerApiProvider({ api: "foo-chat", stream: lazyStreamFoo, ... });
```

The `||=` is the whole lazy mechanism. Calling `lazyStreamFoo()` triggers the import the first time; subsequent calls reuse the resolved promise. No provider module is loaded unless actually needed.

4. Credential detection

`packages/ai/src/env-api-keys.ts` maps `KnownProvider` names to env vars:

```
foo: {
  envVars: ["F00_API_KEY"],
  description: "Foo API key",
}
```

If the env var is present, pi treats it as the provider's credential. If auth flows exist (OAuth, cloud SDK), this file also drives the fallback order.

5. Model generation

`packages/ai/scripts/generate-models.ts` is a build-time script that fetches each provider's model list and writes `packages/ai/src/models.generated.ts`. You add logic to fetch Foo's models (usually an API call to `foo.com/v1/models`), map each to the `Model` type, and include them in the generated output.

The `Model` type has:

```
{
  provider: "foo",
  api: "foo-chat",
  id: "foo-pro-v1",
  contextWindow: 131072,
  cost: {
    input: 3.0,    // $ per million tokens
    output: 12.0,
    cacheRead: 0.3,
    cacheWrite: 3.75,
  },
  // capabilities: supportsThinking, supportsImages, etc.
}
```

The model file is regenerated by `npm run generate-models` in `packages/ai`. It's not checked in from your CI — the author runs it locally when a new provider adds or renames models and commits the result.

6. Tests

AGENTS.md lists eleven test files every new provider must appear in. Every one:

test	what it checks
<code>stream.test.ts</code>	Basic streaming works.
<code>tokens.test.ts</code>	Usage is reported and plausible.
<code>abort.test.ts</code>	Abort mid-stream cleanly.
<code>empty.test.ts</code>	Empty messages and empty responses don't crash.
<code>context-overflow.test.ts</code>	Overflow is detected correctly.
<code>image-limits.test.ts</code>	Images of allowed sizes work; oversized fail gracefully.
<code>unicode-surrogate.test.ts</code>	Unpaired surrogates don't leak into API calls.
<code>tool-call-without-result.test.ts</code>	Missing tool result gets synthetic injection.
<code>image-tool-result.test.ts</code>	Tool results with images survive if supported.
<code>total-tokens.test.ts</code>	Total usage = input + output (cached excluded).
<code>cross-provider-handoff.test.ts</code>	Conversations from other providers feed this one.

Adding a provider is not a one-file change. It's touching each of these, adding a case for the new provider, and running them with a real API key to make sure your provider passes.

7. Coding agent integration

Three touches in `packages/coding-agent/`:

- `src/core/model-resolver.ts` — add a default model id so the CLI can resolve a bare `--provider foo` without a model.
- `src/cli/args.ts` — add the env-var documentation to the `--help` output.
- `README.md` — add a "Foo" section with setup instructions.

8. Documentation

- `packages/ai/README.md` — add Foo to the providers table, document options, auth, env vars.
- `packages/ai/CHANGELOG.md` — add an entry under `[Unreleased]`.

What the pattern means

AGENTS.md calls out one thing worth emphasizing:

Add `export type` re-exports in `packages/ai/src/index.ts` for provider option types that should remain available from the root entry.

Register the provider in `register-builtins.ts` via lazy loader wrappers, **do not statically import provider implementation modules there.**

Violating the "lazy import" rule is the one way to screw up adding a provider. If you statically import `./foo.js` in `register-builtins.ts`, you pay the Foo SDK load cost every time anyone loads pi-ai — in a browser, in serverless, in test. The lazy pattern keeps the overhead zero for providers you don't use.

Non-standard auth

If Foo has weird auth (AWS-style signed requests, OAuth with unusual flows, token exchange with a separate auth endpoint), the convention is:

1. A utility file: `src/providers/foo-utils.ts` with credential detection and request signing.
2. Usage from inside `foo.ts`, not from the general auth layer.

Bedrock does this (`bedrock-utils.ts`); OAuth providers do it via `src/utils/oauth/<provider>.ts`. The pattern is: provider-specific quirks live next to the provider; the general auth layer stays simple.

In one paragraph

Adding a provider touches seven files, each in a specific way. The guardrails — lazy loading via `register-builtins.ts`, eleven tests that new providers must appear in, credential detection in one file — make sure you can't add a provider that silently breaks the bundle size, the test coverage, or the existing providers' cross-handoff. The pattern is explicit, procedural, and well-worn.

Next: testing agents without spending money.

Testing agents with the faux provider

Testing an agent is awkward because the agent's behavior depends on what the LLM does, and what the LLM does depends on a billed-by-the-token API you don't want running in CI. Worse, you want *determinism* — the same test should pass or fail for the same reason every time — but LLMs are probabilistic.

The standard fix, which pi-mono adopted, is a **faux provider**: a stand-in for `pi-ai` that synthesizes deterministic LLM behavior from a script you wrote in the test. No network, no API keys, no variance.

Where tests live

Three layers have distinct test strategies:

`packages/ai` — tests run against real providers

Files under `packages/ai/test/` are Vitest tests that expect real API keys. `stream.test.ts`, `tokens.test.ts`, `cross-provider-handoff.test.ts` — all of them hit real APIs when keys are set.

The root `./test.sh` script filters them based on available keys:

```
# (paraphrased)
if [ -z "$ANTHROPIC_API_KEY" ]; then
  SKIP_ARGS="$SKIP_ARGS --test-name-pattern='^(?!.*anthropic)'"
fi
# ...
```

If you don't have Anthropic keys, Anthropic tests are skipped. The CI is configured with keys for the major providers; a local dev run with no keys silently runs only the provider-agnostic tests.

This is the right default for a library that *has to* talk to real APIs. If Anthropic changes their streaming protocol tomorrow, pi-ai's tests catch it before users notice.

`packages/tui` — Node built-in test runner, no network

pi-tui tests have no external dependencies. They exercise the rendering engine, the diff, the Kitty protocol fallback logic, the overlay compositing, the input decoding. They run in CI every time.

`packages/coding-agent/test/suite/` — faux provider

This is the interesting one.

The faux provider

`packages/coding-agent/test/suite/harness.ts` and its companion files build up a test harness that looks like a miniature pi-ai: same types, same event stream shape, but the "LLM" is a scripted sequence of events the test author writes.

Example (sketchy, but the shape is real):

```
const session = await createTestAgentSession({
  cwd: testDir,
  model: fauxModel,
  scenario: [
    { turn: 1, response: [
      { type: "text", text: "I'll check the readme." },
      { type: "tool_call", name: "read", args: { path: "README.md" } },
    ]},
    { turn: 2, response: [
      { type: "text", text: "The readme looks good. Here's a summary..." },
    ]},
  ],
});

await session.prompt("summarize the readme");
expect(session.messages.length).toBe(4); // user, assistant, toolResult, assistant
```

The harness synthesizes events as if an LLM really produced them. The agent loop processes them just like real events — validation runs, before/after hooks fire, the tool executes for real (so `read` actually reads your test fixture). You get a deterministic end-to-end test of the agent's behavior without an LLM.

What the faux provider cares about

The things tests want to verify:

1. **Tool call validation.** If the scripted tool call has bad arguments, does the agent react correctly?
2. **Permission hooks.** If `beforeToolCall` blocks, does the right thing happen?
3. **Parallel tool execution.** Did both tools run? In what order are results emitted?
4. **Session persistence.** Did the JSONL file get the right entries?
5. **Compaction.** Did compaction trigger at the expected token count? Did the summary message land where expected?
6. **Error handling.** If the scripted turn is an error, did the agent recover (or abort) properly?
7. **Extensions.** Do custom tools and event handlers fire?

The faux provider can script all of these. A scripted "error" turn produces the same events an overflow would. A scripted tool call with bad JSON exercises the validation path. Scripted cascades of tool calls exercise the loop.

Regression tests as issue reproducers

Also from AGENTS.md:

Put issue-specific regressions under `packages/coding-agent/test/suite/regressions/` and name them `<issue-number>-<short-slug>.test.ts`.

An issue filed against pi gets a regression test in this directory. The naming convention

`123-compaction-loses-tool-result.test.ts` links the test to the bug that created it. When someone "fixes" something in the future, the test either keeps passing (good) or starts failing (regression), and the test's name points at the original bug.

Running tests

From AGENTS.md:

Run tests from the package root, not the repo root.

```
cd packages/coding-agent
npx tsx ../../node_modules/vitest/dist/cli.js --run test/suite/my-test.test.ts
```

That absurd command pattern exists because the workspace layout makes `vitest` hard to invoke without a path. The root `./test.sh` runs everything; individual-file runs need the verbose form.

Test harness invariants

AGENTS.md also says:

For `packages/coding-agent/test/suite/`, use `test/suite/harness.ts` plus the faux provider. Do not use real provider APIs, real API keys, or paid tokens.

Bright-line rule. Tests in the suite don't talk to real LLMs. Ever. Not for convenience, not "just this once." Real-LLM tests go in `packages/ai/`. The two worlds stay separated.

The reason: CI shouldn't be billable. If a contributor's PR accidentally introduces a test that hits a real API, a fly-by-night CI run can spend money. Keeping the suite pure means the coding agent's test run can be zero-cost.

What the faux provider doesn't replicate

- **Latency.** Real providers are slow; the faux provider is instant. Tests that care about timing (backpressure behavior, abort mid-stream) need to simulate delays manually.
- **Realistic token counts.** Scripted responses can claim any token count. Compaction tests pick numbers that trigger the logic; they don't mirror real usage patterns.
- **LLM quality effects.** The faux provider can't tell you if the real model would have written a good summary. That's a judgment call that needs real LLM evals (and pi-mono doesn't ship those — the Hugging Face dataset

is the evaluator).

The faux provider is for testing the **framework**, not the **LLM**. That distinction is worth keeping clear.

In one paragraph

Coding-agent tests use a faux provider in `test/suite/harness.ts` that feeds scripted event sequences into the same agent loop a real LLM would — deterministic, free, and end-to-end correct. Regression tests live in `test/suite/regressions/` named by issue number. pi-ai's tests, separately, run against real providers when API keys are set and skip cleanly when they're not. The two layers test different things: the framework in one, the provider integration in the other.

Last chapter of Part V: release engineering.

Release engineering

Releases are the thing every monorepo eventually has to decide about. pi-mono's answer is the shortest possible one: **every package ships together, every time, with the same version number**.

Lockstep versioning

From AGENTS.md:

Lockstep versioning: All packages always share the same version number. Every release updates all packages together.

This is uncommon. Most monorepos use per-package versioning with tools like Changesets or Lerna to track which packages need a bump based on which files changed. Pi-mono does the opposite: if you ship a patch release, all seven packages get patched — even the ones that didn't change in the release window.

The tradeoff:

- **Cost:** noisier npm releases, more version bumps, more changelog entries (some saying "no changes this cycle").
- **Benefit:** the version number is a commit label. `pi-ai@0.67.1` and `pi-coding-agent@0.67.1` are literally from the same commit on main. No compatibility matrix, no wondering which agent version needs which ai version.

For a tightly integrated product like pi, this is the right tradeoff.

No major versions

Also from AGENTS.md:

Version semantics (no major releases):

- `patch`: Bug fixes and new features
- `minor`: API breaking changes

Breaking changes ship as minor bumps while the project is in the 0.x range. This is mild semver abuse but explicit; since you pin with `^0.67.1` you're only locked to patch levels within `0.67.x`. Every minor bump is a conscious upgrade step.

Changelogs in `packages/*/CHANGELOG.md` spell out what changed.

The release scripts

Root `package.json`:

```
"version:patch": "npm version patch -ws --no-git-tag-version && node scripts/sync-versions.js && shx rm -rf node_modules packages/*/node_modules package-lock.json && npm install",
"version:minor": "npm version minor -ws --no-git-tag-version && node scripts/sync-versions.js && shx rm -rf node_modules packages/*/node_modules package-lock.json && npm install",
```

Step by step:

1. `npm version patch -ws --no-git-tag-version` — bumps every workspace package's version in its own `package.json`.
2. `node scripts/sync-versions.js` — walks dependents and updates their `dependencies:` pins to match. So if `pi-coding-agent` depends on `pi-ai`, the new `pi-coding-agent@0.67.2` now has `"@mariozechner/pi-ai": "^0.67.2"`.
3. Nuke `node_modules` and the lockfile.
4. `npm install` to resolve everything fresh.

The script `release.mjs` wraps this with changelog finalization (adds a date to `[Unreleased]` and opens a new `[Unreleased]` section), commits, tags, and publishes.

The `prepublishOnly` guard

```
"prepublishOnly": "npm run clean && npm run build && npm run check"
```

Before any publish, npm runs `prepublishOnly`. The guard:

- Cleans every package.
- Builds everything (in the correct order per the `cd` pipeline).
- Runs Biome + tsgo type check.

If any of that fails, publish aborts. Versions don't ship unless the repo can build and pass lint from scratch.

The changelog discipline

Every package has its own `CHANGELOG.md` with `[Unreleased]` at the top. Contributions add to the appropriate subsection under `[Unreleased]`:

- `### Breaking Changes` — API changes requiring migration
- `### Added` — new features
- `### Changed` — changes to existing functionality

- `### Fixed` — bug fixes
- `### Removed` — removed features

AGENTS.md enforces rules:

Before adding entries, read the full `[Unreleased]` section to see which subsections already exist.

New entries ALWAYS go under `## [Unreleased]` section.

Append to existing subsections, do not create duplicates.

NEVER modify already-released version sections. Each version section is immutable once released.

At release time, the script renames `[Unreleased]` to `[0.67.2]` with the date, and adds a fresh empty `[Unreleased]` at the top.

Attribution rules:

- Internal changes (from issues): `Fixed foo bar ([#123](.../issues/123))`.
- External contributions: `Added feature X ([#456](.../pull/456) by [@user](.../user))`.

The OSS weekend script

There's a separate, unrelated release-adjacent script: `scripts/oss-weekend.mjs`. pi-mono has an explicit "OSS weekend" policy where the issue tracker closes from Monday to the following Monday each week. During OSS weekend, `.github/workflows/oss-weekend-issues.yml` auto-closes new issues from non-maintainers, and `.github/workflows/pr-gate.yml` auto-closes PRs from approved non-maintainers with a weekend message.

The README's top-of-file block gets updated every week by running the oss-weekend script with `--mode=close --end-date=...` or `--mode= open`. It's a small piece of maintenance automation that's not really "release" but lives in the same scripts directory.

Publishing

```
npm run publish
```

Runs `prepublishOnly` then `npm publish -ws --access public`. Every workspace package publishes to npm in parallel. `--access public` because the `@mariozechner` scope is scoped (scoped packages default to private on npm).

`npm run publish:dry` is the same thing with `--dry-run` so you can verify the diff before actually pushing bits to npm.

Tags and Git

The version scripts use `--no-git-tag-version` explicitly; tagging happens in `release.mjs` after the commit. One commit per release, one tag per release, all packages bumped in that one commit.

`git log --oneline` shows the release cadence clearly:

```
e3247c4 book: add preface and chapters 1-14
49ed95e Add Remotion-based architecture walkthrough video
efc58fe Add [Unreleased] section for next cycle
5326452 Release v0.67.1
0fa9539 fix(coding-agent): correct changelog heading for startup notes
```

Every `Release` commit is a clean tag point. Every `[Unreleased]` commit opens the next cycle.

What pi-mono chose not to do

- **No automated release notes from PR titles.** Changelogs are hand-curated. The author prefers prose that names the *why* over auto-generated lists of commit subjects.
- **No beta / rc channel.** The project ships to `latest` directly. Users who want stability pin to a specific version; users who want the latest run `@latest`.
- **No changesets for "only some packages changed."** All packages bump. See above.

In one paragraph

pi-mono releases are lockstep: every package at the same version, every time. The mechanism is `npm version -ws` across the workspace plus a `sync-versions.js` script that rewrites inter-package dependencies; `prepublishOnly` enforces a clean build and type check; changelogs are hand-curated with a formal `[Unreleased]` → `[X.Y.Z]` promotion. It's deliberately simple, trusting that the author would rather bump all seven packages than maintain an inter-package compatibility matrix.

The core chapters end here. Three appendices follow.

A P P E N D I X

Reference

Appendix A — Glossary

Terms that come up repeatedly in pi-mono, in one place.

Agent

A class (`pi-agent-core`) that orchestrates a loop of LLM calls and tool executions. Holds mutable state: messages, tools, model, streaming status, pending tool calls, queues.

AgentMessage

A union of `UserMessage`, `AssistantMessage`, `ToolResultMessage`, and custom message types. The entries in an agent's conversation history.

AgentSession

pi-coding-agent's session wrapper. Extends `Agent` with session-specific concerns: JSONL persistence, compaction triggers, custom message support, tool registry.

AgentSessionRuntime

Owner of the services bound to a working directory: session manager, auth storage, extensions, tools, cwd. Torn down and rebuilt on `/switch` or `/fork`.

agentLoop, agentLoopContinue

Two lower-level entry points in pi-agent-core. `agentLoop` takes new user messages; `agentLoopContinue` resumes without a new message (used for retrying after tool errors or aborts).

Api

A provider's API identifier — `"anthropic-messages"`, `"openai-completions"`, etc. Multiple providers can share an API (e.g., many OpenAI-compatible providers use `"openai-completions"`).

AssistantMessageEvent

The discriminated union of streaming events emitted by pi-ai. Types include `start`, `text_delta`, `toolcall_delta`, `thinking_delta`, `done`, `error`.

AssistantMessageEventStream

An `EventStream` of `AssistantMessageEvent`s returned by pi-ai's `stream()`.

Backpressure

The property of a stream consumer slowing down the producer. In pi-mono, `agentLoop` does not apply backpressure (producer keeps going regardless of consumer speed); the `Agent` class does (awaits all subscribers before advancing).

beforeToolCall, afterToolCall

Hooks in pi-agent-core's `AgentLoopConfig`. `beforeToolCall` is a permission gate that can block execution; `afterToolCall` can replace a tool's result.

Branch (session)

A path from the session's root entry to a leaf, defined by following `parentId` pointers. Forking creates a new leaf with a different `parentId`.

Compaction

Summarizing old messages and inserting a `compaction` entry into the session tree, so future LLM calls see the summary in place of the old history.

Context

pi-ai's `{ systemPrompt?, messages, tools? }` — the full state needed to make an LLM call. Intentionally mutable; accumulates over turns.

CSI 2026

ANSI escape sequence (`\x1b[?2026h`) that tells the terminal to render atomically instead of showing intermediate state. pi-tui wraps every render in it.

Custom message

A JSONL session entry of type `custom_message` written by extensions. Arbitrary content; rendered by registered renderers.

Differential rendering

pi-tui's strategy of diffing rendered line arrays and emitting only the changed lines.

EventStream

A generic queue+promise hybrid (`packages/ai/src/utils/event-stream.ts`). Every streaming layer in pi-mono uses one.

Extension

A TypeScript file loaded at startup from `~/.pi/extensions/` or `./.pi/extensions/`. Runs in-process with no sandbox. Can register tools, commands, shortcuts, providers, renderers, event handlers.

Faux provider

The test harness in `packages/coding-agent/test/suite/harness.ts` that stands in for pi-ai, producing deterministic scripted event streams so tests run without real API keys.

/fork

Command that creates a new session file branching from the current entry. Original session is untouched; new session has its own JSONL file and genealogy link.

Hook (before / after)

See `beforeToolCall` / `afterToolCall`.

JSONL

Line-delimited JSON. pi-mono uses it for session files, log files, pi-ai test fixtures, and the Hugging Face published sessions.

jiti

A TypeScript runtime loader ([@mariozechner/jiti](#)). Used by pi-coding-agent to import extensions without a compile step.

Kitty keyboard protocol

An extended keyboard reporting protocol supported by Kitty, Ghostty, WezTerm, and others. pi-tui queries for it on startup with a 150 ms timeout before falling back to xterm `modifyOtherKeys` mode 2.

KnownProvider

pi-ai's type listing supported providers by name — `"anthropic"`, `"openai"`, etc.

Leaf

The entry at the tip of the current session branch. Where new entries get appended as children.

Lockstep versioning

pi-mono's release policy: every package always shares the same version number.

MCP

Model Context Protocol. External tool-server protocol. **Not built in to pi**. Implemented as an extension.

MEMORY.md

A file in a workspace that gets appended to the system prompt. Used for "always remember this" facts. Agents can read and write to their own MEMORY.md.

Mode

How pi-coding-agent is I/O-configured: `interactive`, `print`, `json`, or `rpc`.

Overflow (context)

LLM error meaning the request was too big for the context window. pi-ai detects it via regex patterns; pi-coding-agent triggers compaction to recover.

partial-json

The npm package used by `parseStreamingJson()` to parse incomplete JSON during tool-call streaming.

proper-lockfile

The npm package used by pi-coding-agent to serialize concurrent writes to session files and auth storage.

RPC mode

JSON-RPC over stdin/stdout. Used by IDE extensions and pi-mom (when not embedded).

Session

pi-coding-agent's conversation state, persisted as a JSONL file.

SKILL.md

A Markdown file with frontmatter describing a named prompt template that can be invoked via `/skill:name`. Stateless; not executable.

Steering queue

Messages the user sends while the agent is already running. Drained between turns. Mode `"all"` or `"one-at-a-time"`.

Streaming JSON

Parsing not-yet-complete JSON during tool-call deltas. See `parseStreamingJson()`.

TSchema

TypeBox schema type (`@sinclair/typebox`). Used everywhere pi-mono describes a tool's input.

Thinking level

Controls extended-thinking behavior: `"off"`, `"minimal"`, `"low"`, `"medium"`, `"high"`, `"xhigh"`.
Provider-specific semantics; not all providers support all levels.

Tool

In pi-ai: `{ name, description, parameters: TSchema }`. In pi-agent-core: `AgentTool` extends `Tool` with `label`, `prepareArguments?`, `execute`.

tsgo

TypeScript's native Go-port compiler. Used across pi-mono instead of `tsc` for speed.

Turn

One LLM call and whatever tool executions it triggers. Marked by `turn_start` / `turn_end` events.

vLLM

An open-source LLM inference server. pi-pods deploys and manages vLLM instances on remote GPUs.

Appendix B — File map

A reader's map of the repo. Not exhaustive; annotated with what each important file contains.

Repo root

```
pi-mono/
├─ AGENTS.md           # dev rules for humans and agents. Read this first.
├─ CONTRIBUTING.md     # outward-facing contribution guide.
├─ README.md           # project overview, package table, OSS weekend banner.
├─ biome.json          # the only formatter/linter config.
├─ package.json        # workspace definition, build/release scripts.
├─ pi-test.sh          # run pi from source anywhere in the system.
├─ test.sh             # run tests, skipping LLM-dependent ones without keys.
├─ tsconfig.base.json  # shared TypeScript config.
├─ .pi/                # per-repo pi config. extensions, skills, session data.
├─ scripts/
│   └─ release.mjs     # orchestrates a release cycle.
│   └─ sync-versions.js # rewrites inter-package dep pins after a version bump.
│   └─ oss-weekend.mjs # toggles OSS weekend banner + CI gates.
│   └─ profile-coding-agent-node.mjs # CPU profiler harness.
└─ packages/           # the real content.
```

packages/ai ([@mariozechner/pi-ai](#))

```

packages/ai/
├─ src/
│   │   └─ index.ts           # public entry. re-exports types.
│   │   └─ types.ts          # Context, Message, Tool, Api, KnownProvider, events.
│   │   └─ api-registry.ts    # Map<Api, RegisteredApiProvider>, the runtime registry.
│   │   └─ models.ts         # calculateCost, getModel, getModels.
│   │   └─ models.generated.ts # auto-generated. Per-provider model lists with costs.
│   │   └─ env-api-keys.ts    # env var → provider credential map.
│   │   └─ providers/
│   │       │   └─ register-builtins.ts # lazy registration of every built-in provider.
│   │       │   └─ transform-messages.ts # cross-provider message transforms.
│   │       │   └─ anthropic.ts
│   │       │   └─ openai-completions.ts
│   │       │   └─ openai-responses.ts
│   │       │   └─ openai-codex-responses.ts
│   │       │   └─ google-gemini.ts
│   │       │   └─ google-gemini-cli.ts
│   │       │   └─ google-vertex.ts
│   │       │   └─ google-antigravity.ts
│   │       │   └─ amazon-bedrock.ts
│   │       │   └─ mistral.ts
│   │       │   └─ groq.ts
│   │       │   └─ cerebras.ts
│   │       │   └─ xai.ts
│   │       │   └─ openrouter.ts
│   │       │   └─ ai-gateway.ts
│   │       │   └─ minimax.ts
│   │       │   └─ azure-openai.ts
│   │       │   └─ copilot.ts
│   │       │   └─ opencode-zen.ts
│   │       │   └─ opencode-go.ts
│   │       │   └─ kimi.ts
│   │       └─ bedrock-utils.ts, oauth/ # per-provider auth helpers.
│   └─ utils/
│       │   └─ event-stream.ts # EventStream<T, R> queue+promise hybrid.
│       │   └─ overflow.ts     # isContextOverflow, regex patterns.
│       │   └─ json-parse.ts   # parseStreamingJson for partial JSON.
│       │   └─ validation.ts    # validateToolCall via AJV.
│       │   └─ sanitize-unicode.ts # surrogate pair sanitizer.
│       └─ oauth/index.ts      # OAuth provider registry + refresh.
├─ scripts/
│   └─ generate-models.ts      # fetches model lists, writes models.generated.ts.
└─ test/
    │   └─ stream.test.ts
    │   └─ tokens.test.ts
    │   └─ abort.test.ts
    │   └─ empty.test.ts
    │   └─ context-overflow.test.ts
    │   └─ image-limits.test.ts
    │   └─ unicode-surrogate.test.ts
    │   └─ tool-call-without-result.test.ts
    │   └─ image-tool-result.test.ts
    │   └─ total-tokens.test.ts
    └─ cross-provider-handoff.test.ts

```

packages/agent (@mariozechner/pi-agent-core)

```
packages/agent/  
└─ src/  
    │ index.ts           # public entry: Agent, agentLoop, types.  
    │ types.ts           # AgentMessage, AgentTool, AgentLoopConfig, hooks.  
    │ agent.ts           # Agent class – mutable state machine, queue modes.  
    │ agent-loop.ts      # agentLoop, agentLoopContinue – core loop.  
    └─ proxy.ts          # streamProxy for routing through a backend.
```

packages/tui (@mariozechner/pi-tui)

```
packages/tui/  
└─ src/  
    │ index.ts  
    │ tui.ts             # the TUI class and render engine.  
    │ terminal.ts        # ProcessTerminal abstraction, Kitty/xterm setup.  
    │ terminal-image.ts  # Kitty/iTerm2 image encoding.  
    │ stdin-buffer.ts    # escape sequence reassembler.  
    │ keys.ts            # semantic key decoder.  
    │ keybindings.ts     # KeybindingsManager.  
    │ utils.ts           # graphemeWidth, AnsiCodeTracker, caches.  
    │ autocomplete.ts    # file path / command autocomplete source.  
    └─ components/  
        │ text.ts  
        │ input.ts  
        │ editor.ts      # multi-line editor, undo, kill ring, paste markers.  
        │ markdown.ts  
        │ loader.ts  
        │ select-list.ts  
        │ settings-list.ts  
        │ image.ts  
        │ box.ts  
        └─ container.ts
```

packages/coding-agent (@mariozechner/pi-coding-agent)

```

packages/coding-agent/
├── src/
│   ├── cli.ts                # process.title, delegate to main.
│   ├── main.ts              # orchestration: args, modes, sessions.
│   ├── index.ts             # SDK export.
│   ├── cli/
│   │   └── args.ts          # argument parser with compound syntax.
│   ├── core/
│   │   ├── agent-session.ts  # extends Agent with session concerns.
│   │   ├── agent-session-runtime.ts # cwd-bound services wrapper.
│   │   ├── session-manager.ts # JSONL append-only persistence, tree walk.
│   │   ├── model-resolver.ts  # resolve --model flag to Model with fallback.
│   │   ├── auth-storage.ts    # ~/.pi/auth.json I/O with proper-lockfile.
│   │   ├── settings-manager.ts # ~/.pi/settings.json + ./pi/settings.json.
│   │   ├── resource-loader.ts # discover extensions/skills/themes.
│   │   ├── skills.ts          # SKILL.md parser.
│   │   ├── sdk.ts             # createAgentSession factory for embedders.
│   │   ├── compaction/
│   │   │   ├── compaction.ts
│   │   │   └── utils.ts      # SUMMARIZATION_SYSTEM_PROMPT, token estimators.
│   │   ├── extensions/
│   │   │   ├── loader.ts     # jiti-based extension loading.
│   │   │   ├── runner.ts     # extension lifecycle (init, bind, destroy).
│   │   │   └── index.ts      # extension API surface.
│   │   └── tools/
│   │       ├── bash.ts
│   │       ├── read.ts
│   │       ├── write.ts
│   │       ├── edit.ts
│   │       ├── grep.ts
│   │       ├── find.ts
│   │       ├── ls.ts
│   │       └── index.ts
│   └── modes/
│       ├── interactive/      # TUI-based mode. Many components.
│       ├── print-mode.ts     # single-shot.
│       └── rpc/
│           ├── rpc-mode.ts   # stdin/stdout JSON-RPC loop.
│           └── rpc-types.ts  # command and event types.
└── test/
    ├── suite/
    │   ├── harness.ts        # faux provider + test helpers.
    │   └── regressions/      # issue-specific regressions.

```

packages/mom

```

packages/mom/
└─ src/
    ├── main.ts           # entry, channel state, run loop.
    ├── slack.ts         # Socket Mode handler.
    ├── agent.ts         # AgentRunner, system prompt, event plumbing.
    ├── events.ts        # file-system event watcher.
    ├── sandbox.ts       # host vs. docker executor.
    └─ store.ts          # ChannelStore, dedup window.

```

packages/web-ui

```

packages/web-ui/
└─ src/
    ├── index.ts
    ├── ChatPanel.ts      # top-level composition.
    ├── components/
    │   ├── AgentInterface.ts
    │   ├── MessageList.ts
    │   ├── StreamingMessageContainer.ts
    │   ├── MessageEditor.ts
    │   ├── SandboxedIframe.ts  # the large one. postMessage bridge.
    │   ├── Messages.ts
    │   ├── ModelSelector.ts
    │   ├── SettingsDialog.ts
    │   ├── SessionListDialog.ts
    │   ├── CustomProviderDialog.ts
    │   ├── ApiKeyPromptDialog.ts
    │   └─ (artifact renderers)
    ├── storage/
    │   ├── app-storage.ts      # IndexedDB wrapper.
    │   ├── sessions-store.ts
    │   ├── settings-store.ts
    │   ├── custom-providers-store.ts
    │   └─ provider-keys-store.ts
    ├── tools/
    │   └─ artifacts/artifacts.ts
    └─ utils/
        ├── proxy-utils.ts     # CORS bypass.
        └─ example/           # working demo app.

```

packages/pods

```
packages/pods/
└─ src/
    ├── cli.ts           # command dispatcher.
    ├── config.ts        # ~/.pi/pods.json I/O.
    ├── types.ts         # Pod, GPU, Model interfaces.
    ├── model-configs.ts # catalog of supported models.
    ├── ssh.ts           # ssh helper.
    └─ commands/
        ├── pods.ts      # pod setup / list / active.
        ├── models.ts    # start / stop / logs / list.
        └─ prompt.ts     # pi agent <pod> shortcut.
```


Appendix C — The rules pi-mono seems to follow

A condensed version of the intuitions scattered throughout the book. Not a style guide; closer to the shape of the codebase's taste. Lifted from AGENTS.md where applicable, and inferred from the code elsewhere.

Architecture

1. **Start with the minimum. Add nothing until a concrete use forces you to.** The agent ships with `read`, `write`, `edit`, `bash` and no more.
2. **Everything beyond the minimum is an extension, not a feature.** MCP, plan mode, subagents, memory files — all extensions, none in the core.
3. **One way to do things.** Global `~/.pi/settings.json`, project `./.pi/settings.json`, deep merge, project wins. That's the whole config system.
4. **The event stream is the contract.** Every layer emits an async iterator of discriminated events. Errors travel as events; only caller mistakes throw.
5. **Sessions are JSONL files.** Nothing more sophisticated than append-only line-delimited JSON.
6. **Cross-provider fidelity is a feature.** Engineering time goes into making a conversation started on one provider feed another provider correctly.

Code

7. **No `any`.** If the compiler complains, fix the code or upgrade the dependency — don't widen the type.
8. **Never use inline imports or dynamic imports for types.** Standard top-level imports only.
9. **Check `node_modules` for external API types. Don't guess.**
10. **Write no comments by default.** Add one only when the *why* is non-obvious. Don't describe what the code does; the code does that.
11. **Extract a function after the third copy, not before.** Three similar lines is better than a premature abstraction.
12. **No `try` / `catch` for errors that aren't really exceptions.** Put them in the event stream instead.

Input and UI

13. **Never hardcode key checks.** All keybindings go through `KeybindingsManager` with configurable names.
14. **Render width is a hard contract.** A component returning a line wider than the provided width is a bug, and pi-tui hard- fails.
15. **Line-level diff is enough.** Cell-level diffing is not worth the complexity.

16. **Wrap every render in CSI 2026.** Modern terminals understand it; old ones ignore it cleanly.

Tests

17. **Real API tests and framework tests never mix.** pi-ai tests hit real providers (when keys are set); coding-agent tests use a faux provider and never touch the network.

18. **Issue regressions get their own test file, named by issue number.** `123-short-slug.test.ts`.

19. **Tests run from their package root.** Not from the repo root.

Git and releases

20. **Lockstep versions.** Every package always shares the same version. No exceptions.

21. **minor is breaking, patch is additive.** No major releases until we leave 0.x. Breaking changes are deliberately minor bumps.

22. **Changelogs are hand-curated.** Under `[Unreleased]` until release day; frozen thereafter.

23. **prepublishOnly guards every release.** Clean + build + check, or nothing ships.

24. **Only commit files you actually changed.** Never `git add -A` or `git add .`. The repo is deliberately structured for parallel work; sweeping stages sweep up other people's work.

25. **Never force push, never `--no-verify`.** Full stop.

Extensions and trust

26. **Extensions run unsandboxed.** The trust model is "you installed it." Don't install extensions you don't trust.

27. **Backward compatibility is opt-in.** Don't add compat shims unless asked. Don't delete load-bearing code without asking.

Process and communication

28. **Always read all comments on an issue before acting.**

29. **Write the full comment to a temp file before posting to GitHub.** Never pass multi-line markdown via `--body` in shell.

30. **Post exactly one final comment per task unless the user asked for multiple.**

31. **Close issues via commit message (`fixes #123`).**

32. **Keep comments technical and in the user's tone.** No filler.

The overarching rule

33. **The repo is sized to fit in one head.** Every decision above is downstream of this. If something makes the repo feel smaller and more readable, it's probably right. If it makes the repo more general, more abstracted, more

"future-proof," it's probably wrong.

If this list feels prescriptive, that's because lists of rules always do. In practice the code reads less like "we followed these rules" and more like "these rules describe the way we ended up writing code." The author wrote what felt right, and this is what felt right often enough to become habit.

Read the code. It's the real reference.